Moduły analizy TestGEN (AI)

Spis treści

- Moduły analizy — indeks
  - Przegląd
  - Analiza kodu źródłowego (Epic 1)
  - Analiza dokumentacji (Epic 2)
  - Analiza UML i GUI (Epic 3)
  - Analiza bezpieczeństwa (Epic 4)
  - Analiza luk funkcjonalnych (Epic 5)
  - Granice modułów
- Dokumentacja modułów AI TestGEN (M1–M6)
  - Spis treści
  - 1. Wprowadzenie
  - 2. M1 — Analiza kodu źródłowego
  - 3. M2 — Analiza dokumentacji
  - 4. M3 — Analiza UML
  - 5. M4 — Analiza GUI
  - 6. M5 — Generowanie testów
  - 7. M6 — Analiza bezpieczeństwa
  - 8. Porównanie zbiorcze modułów
  - 9. LLM Gateway — warstwa abstrakcji
  - 10. Optymalizacja i koszty
  - Powiązane dokumenty
- Analiza dokumentacji — pipeline natywny (Epic 2)
  - Etapy pipeline'u
  - Architektura asynchroniczna
  - Klasyfikacja modułów w `AnalysisResult.module`
  - 1. Preprocessing (`DocumentPreprocessor`)
  - 2. Klasyfikacja (`DocumentClassifier`)
  - 3. Ekstrakcja wymagań (`RequirementExtractor`)
  - 4. Glosariusz (`GlossaryExtractor` + `TerminologyMapper`)
  - 5. Detekcja niespójności (`InconsistencyDetector`)
  - 6–8. Moduły zależne (tylko do odczytu)
  - Workflow i reparse
  - Powiązane
- Formaty dokumentacji — macierz możliwości (Epic 2)
  - Możliwości ekstrakcji per format
  - Detekcja nagłówków per format
  - Limity

- Znane ograniczenia
- Fabryka parserów

Analiza AST kodu źródłowego (Epic 1)
- Obsługiwane języki
- Ekstrahowane struktury (`logic_structures`)
- Właściwości
- Buforowanie i diff
- Powiązane

Benchmarki i golden data (P1)
- Koncepcja
- Moduł dokumentacji (Epic 2)
- Moduł kodu (Epic 1)
- Uruchamianie
- Aktualizacja

Analiza regresji na podstawie diff AST (Epic 1)
- Diff struktur
- Selekcja testów regresyjnych
- Testy regresyjne i golden data
- Powiązane

Generowanie przypadków testowych — silnik natywny (Epic 1)
- Klasyfikacja przypadków testowych
- Dane testowe i mocki
- Szablony per framework
- Workflow
- Powiązane

Generowanie kodu testowego per framework (Epic 1)
- Frameworki jednostkowe/integracyjne
- Frameworki E2E/mobilne (ADR-007)
- Rejestr frameworków
- Właściwości
- Powiązane

Analiza GUI — pipeline natywny CV/OCR (Epic 3)
- Etapy pipeline'u
- Preprocessing (`component_detector._cv2_regions`)
- OCR (`ocr_service.py`)
- Detekcja komponentów (`component_detector.classify_region`)
- Analiza layoutu (`layout_analyzer.py`)
- Regresja wizualna (3.10, `visual_regression.py`)
- Generowanie PNG SikuliX (domknięcie 3-13)
- Moduł → serwis → story
- Ograniczenia
- Powiązane

Generowanie testów z UML/GUI — pipeline natywny (Epic 3)
- Etapy pipeline'u
- Rekonstrukcja przepływów (3.7 / 3.9)
- Mapowanie Gherkin (3.11, `gherkin_mapper.py`) — domknięcie 3-11

- Codegen (3.12 / 3.13)
- Moduł → serwis → story
- Powiązane

Macierz śledzenia UML/GUI — pipeline natywny (Epic 3)
- Etapy pipeline'u
- Spójność typów post-3-1b (domknięcie 3-15)
- Provenance (domknięcie 3-15)
- Semantyka pokrycia — Gherkin-proxy (AC4, domknięcie 3-15)
- Eksport (AC5, domknięcie 3-15)
- Moduł → serwis → story
- Powiązane

Modele danych — moduł UML/GUI (Epic 3)
- Diagram ERD (PlantUML)
- Enumy (wartości live)
- Tabele (kluczowe pola)
- Schematy Pydantic (przykłady)
- Golden data (3-14)
- Powiązane

Parsowanie UML — pipeline natywny (Epic 3)
- Etapy pipeline'u
- Formaty (macierz: format × typ diagramu)
- Wykrywanie formatu i typu
- Adapter visual→structural (domknięcie 3-1b)
- compare-code (3.4 + domknięcie 3-1b)
- Ograniczenia (udokumentowane, szczere)
- Moduł → serwis → story
- Powiązane

Architektura modułów bezpieczeństwa — przegląd
- 1. Cel modułu
- 2. Diagram pipeline'u
- 3. Moduły i ich odpowiedzialności
- 4. Wspólne typy wyników
- 5. Przepustowość i ograniczenia
- 6. Powiązane dokumenty

SAST engine — silnik statycznej analizy
- 1. Cel
- 2. Interfejs
- 3. Algorytm (ścieżka native)
- 4. Korekty pewności (confidence)
- 5. Mapa override severity
- 6. Wejścia i wyjścia
- 7. Zależności
- 8. Wydajność

Taint analyzer — analiza przepływu danych
- 1. Cel
- 2. Interfejs

3. Struktury danych
4. Algorytm
5. Detekcja przypisań
6. Nieadekwatna sanityzacja
7. Zależności

Dependency scanner — skaner zależności (SCA)

1. Cel
2. Obsługiwane ekosystemy i parsery
3. Interfejs
4. Struktury danych
5. Algorytm
6. Pasma severity (CVSS)
7. Tryb offline
8. Zależności

Config scanner — skaner konfiguracji infrastruktury

1. Cel
2. Klasyfikacja plików
3. Interfejs
4. Reguły per technologia
5. Algorytm orkiestratora
6. Limity

DAST runner — dynamiczna analiza (OWASP ZAP)

1. Cel
2. Interfejs
3. Struktury danych
4. Algorytm
5. Sanityzacja danych wrażliwych
6. Mapowanie ZAP → CWE
7. Stałe
8. Zależności

Agregator wyników i risk engine

CZĘŚĆ A — FindingAggregator
CZĘŚĆ B — RiskEngine
CZĘŚĆ C — CVSSCalculator

Modele danych — moduł bezpieczeństwa

1. Enumy
2. Diagram ERD (uproszczony)
3. Tabela SecurityScan
4. Tabela SecurityFinding
5. Tabela CVEDatabase

Katalog reguł — SQL Injection (CWE-89)

Opis klasy podatności
Reguły
Przykład podatny (Python)
Przykład naprawiony (Python)
Rekomendacja

- Katalog reguł — Cross-Site Scripting (CWE-79)
  - Opis klasy podatności
  - Reguły
  - Przykład podatny (JavaScript, DOM-based)
  - Przykład naprawiony
  - Rekomendacja
- Katalog reguł — CSRF (CWE-352)
  - Opis klasy podatności
  - Reguły
  - Przykład podatny (Django)
  - Przykład naprawiony
  - Rekomendacja
- Katalog reguł — Broken Authorization / IDOR (CWE-862/639)
  - Opis klasy podatności
  - Reguły
  - Przykład podatny (Django, IDOR)
  - Przykład naprawiony
  - Rekomendacja
- Katalog reguł — walidacja wejścia i injection
  - Reguły
  - Przykład podatny (command injection)
  - Przykład naprawiony
  - Rekomendacje skrótem
- Katalog reguł — Hardcoded Secrets (CWE-798)
  - Opis klasy podatności
  - Reguły
  - Przykład podatny
  - Przykład naprawiony
  - Rekomendacja
- Katalog reguł — macierz języków
  - Macierz reguła → języki
  - Podsumowanie pokrycia
  - Rejestr reguł (`rules/registry.py`)
- Katalog reguł — definicje taint (sources / sinks / sanitizers)
  - Struktury bazowe (`base.py`)
  - Python
  - JavaScript / TypeScript
  - Java
  - C#
  - C / C++
  - Adekwatność sanitizera
- Pipeline DAST — integracja OWASP ZAP
  - 1. Architektura integracji
  - 2. Cykl życia skanu
  - 3. Konfiguracja
  - 4. Wywołanie

5. Endpoint API
6. Limity i bezpieczeństwo

Pipeline DAST — payloady i fuzzing

1. Mechanizm aktywnego skanu
2. Typy ataków wykrywanych dynamicznie
3. Limity wydajności
4. Faza spider
5. Bezpieczeństwo payloadów

Pipeline DAST — autentykacja

1. Obsługiwane metody
2. Konfiguracja w ZAP
3. Przykład (token)
4. Przykład (form)
5. Testy kontroli dostępu

Pipeline DAST — parsowanie wyników

1. Od alertu ZAP do SecurityFinding
2. DASTFinding
3. Mapowanie severity
4. Mapowanie CWE
5. Sanityzacja request/response
6. Korelacja z SAST

Pipeline CVE — źródła danych

1. Cel
2. Źródła danych
3. Parametry NVD
4. Struktury danych
5. Metody pobierania
6. Deduplikacja i scalanie
7. Persistencja

Pipeline CVE — harmonogram i konfiguracja

1. Harmonogram (Celery Beat)
2. Konfiguracja zadania (`cve_tasks.py`)
3. Rate limiting
4. Orkiestracja głównego przebiegu
5. Walidacja na danych golden

Pipeline CVE — auto-generowanie reguł SAST

1. Cel
2. Generowanie
3. Parsowanie szkicu
4. Walidacja na golden data
5. Bezpieczeństwo procesu

Pipeline CVE — auto re-skan projektów i powiadomienia

1. Wykrywanie dotkniętych projektów
2. Wyzwolenie re-skanu
3. Powiadomienia
4. Przepływ end-to-end

5. Logowanie przebiegu

Architektura modułu luk funkcjonalnych — przegląd

1. Cel modułu
2. Diagram pipeline'u
3. Moduły i ich odpowiedzialności
4. Tryb pracy
5. Model danych
6. Przepustowość i ograniczenia
7. Powiązane dokumenty

Comparator — fuzzy matching wymagań z kodem

1. Odpowiedzialność
2. Algorytmy dopasowania
3. Kombinacja wyników
4. Progi klasyfikacji
5. Struktury danych
6. API
7. Testowanie
8. Powiązane dokumenty

Klasyfikator 4 typów luk

1. Odpowiedzialność
2. Cztery typy luk
3. Reguły klasyfikacji
4. Mapowanie severity
5. Context excerpts
6. Konfiguracja
7. Testowanie
8. Powiązane dokumenty

Compliance engine

1. Odpowiedzialność
2. Poziomy scoringu
3. Wzór scoringu
4. Przykład
5. Struktury wynikowe
6. Endpointy API
7. Eksport CSV
8. Frontend
9. Testowanie
10. Powiązane dokumenty

Recommendation engine

1. Odpowiedzialność
2. Mapowanie priorytetów
3. Rekomendacje szablonowe
5. Szacowanie wysiłku (effort_estimate)
6. Persystencja
7. Endpointy API
8. Frontend

9. Testowanie
10. Powiązane dokumenty

Skeleton generator

1. Odpowiedzialność
2. Obsługiwane języki
3. Mechanizm
4. Szablony Jinja2
5. Walidacja tree-sitter
6. Persystencja
7. Endpointy API
8. Frontend
9. Testowanie
10. Powiązane dokumenty

Integracje z trackerami (Jira, Azure DevOps)

1. Odpowiedzialność
2. Struktura danych
3. Szyfrowanie poświadczeń
4. JiraService
5. AzureDevOpsService
6. Sync flow
7. Deduplikacja
8. Endpointy API
9. Frontend
10. Bezpieczeństwo
11. Testowanie
12. Powiązane dokumenty

Modele danych (ERD)

1. Diagram
2. gap_findings
3. code_errors
4. recommendations
5. compliance_snapshots
6. integration_configs + integration_sync_logs
7. Migracje
8. RLS
9. Powiązane dokumenty

Moduły analizy — indeks

> Dokumentacja modułów generujących przypadki testowe i wyniki analiz.

Przegląd

- ai-modules.md — **kompletna dokumentacja modułów AI (M1-M6):** dual-path engine, prompty LLM, benchmarki, golden data, porównanie native vs LLM vs hybrid, koszty, optymalizacja

Analiza kodu źródłowego (Epic 1)

analiza-kodu/ — ast-analysis (parsowanie AST 11+ językami, tree-sitter) · generowanie przypadków testowych · generowanie kodu testów E2E (Playwright/Cypress/Selenium) · analiza regresji · benchmarki i golden data

Analiza dokumentacji (Epic 2)

analiza-dokumentacji/ — analiza dokumentów (segmentacja, wymagania, glosariusz, niespójności; PDF/DOCX/OCR) · formaty wejściowe

Analiza UML i GUI (Epic 3)

analiza-uml-gui/ — parsowanie UML (PlantUML/XMI/SVG+OCR) · analiza GUI (zrzuty, detekcja komponentów) · generowanie testów · traceability · modele danych

Analiza bezpieczeństwa (Epic 4)

bezpieczenstwo/ — architektura modułu (overview: SAST, taint, zależności, konfiguracja, DAST, agregator ryzyka) · katalog reguł (SQLi, XSS, CSRF, authz, walidacja, sekrety, taint) · pipeline DAST (ZAP, payloady, autentykacja) · pipeline CVE (źródła, harmonogram, auto-reguły, rescan)

Poradnik użytkownika modułu → ../user-guide/bezpieczenstwo/

Analiza luk funkcjonalnych (Epic 5)

luki-funkcjonalne/ — overview: comparator (dopasowanie wymagań↔kod), klasyfikator 4 typów luk, compliance scoring, rekomendacje, skeleton generator, integracje (Jira/Azure DevOps), modele danych

Poradnik użytkownika modułu → ../user-guide/luki-funkcjonalne/

Granice modułów

- Analiza kodu (E1) dostarcza struktury logiczne wykorzystywane przez porównanie kod↔dokumentacja (E5) i traceability (E3).
- Analiza dokumentacji (E2) wyciąga wymagania — wejście modułu luk (E5).
- Bezpieczeństwo (E4) ma własne tabele wyników (`security_findings`), niezależne od `gap_findings` (E5).

Dokumentacja modułów AI TestGEN (M1–M6)

> **Wersja:** 1.0 · **Ostatnia aktualizacja:** 2026-09 · **Grupa docelowa:** Deweloperzy platformy, architekci AI

Spis treści

1. Wprowadzenie
2. M1 — Analiza kodu źródłowego
3. M2 — Analiza dokumentacji
4. M3 — Analiza UML
5. M4 — Analiza GUI
6. M5 — Generowanie testów
7. M6 — Analiza bezpieczeństwa
8. Porównanie zbiorcze modułów
9. LLM Gateway — warstwa abstrakcji
10. Optymalizacja i koszty

---

1. Wprowadzenie

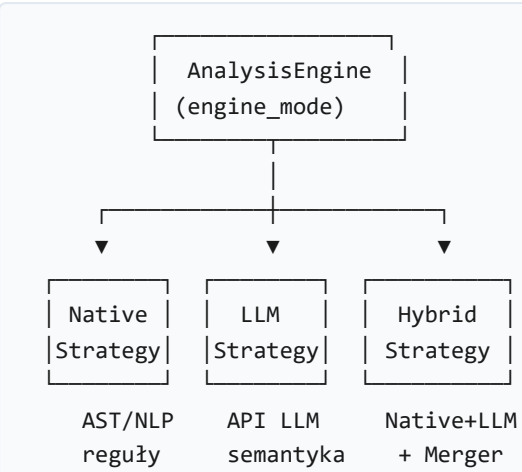
1.1. Cel dokumentu

Dokument opisuje sześć modułów AI platformy TestGEN z perspektywy dewelopera:

- Architektura dual-path (native vs LLM vs hybrid)
- Prompty systemowe i użytkownika z parametrami
- Benchmarki jakości (golden test data, metryki)
- Rekomendacje doboru trybu silnika
- Znane ograniczenia i edge case'y

1.2. Architektura dual-path

Każdy moduł analityczny implementuje wzorzec **dual-path engine** (patrz ADR-006, architektura C4 Level 4):



Tryb	Opis	Koszt	Opóźnienie
`native`	Deterministyczny — AST, NLP, reguły, szablony	0 PLN	< 5 s
`llm`	Semantyczny — model językowy (Anthropic/OpenAI)	0,05–0,50 PLN/analiza	5–30 s
`hybrid`	Native + LLM + scalenie wyników (ResultMerger)	Jak LLM	5–35 s

1.3. LLM Providers

Dostawca	Model domyślny	Model zapasowy	Koszt (input / output per 1M tokenów)
Anthropic	`claude-sonnet-4-20250514`	—	$3,00 / $15,00
OpenAI	`gpt-4o`	`gpt-4o-mini` (fallback na 5xx)	$5,00 / $15,00
OpenAI (mini)	`gpt-4o-mini`	—	$0,15 / $0,60

Dostawca konfigurowany zmienną środowiskową `LLM_PROVIDER`. Domyślny: `anthropic`.

1.4. Prompt Manager

System zarządzania promptami (`services/analysis/prompt_manager.py`):

- Prompty ładowane z plików YAML (`backend/app/prompts/*.yaml`)
- Renderowanie zmiennych via Jinja2
- Wersjonowanie: `v1` (stable) / `v2` (latest)
- Routing A/B: `sha256(name:tenant_id) % 100 < ab_test_percentage`
- Pozwala na ewolucję promptów bez zmian w kodzie

---

2. M1 — Analiza kodu źródłowego

2.1. Cel i zakres

Moduł M1 parsuje i analizuje kod źródłowy — ekstrakcja struktury (klasy, funkcje, importy), złożoności cyklomatycznej, zależności modułów, przepływów danych i logiki biznesowej.

Lokalizacja w kodzie: `backend/app/services/analysis/`

2.2. Obsługiwane języki

Język	Parser	Adapter	Status
Python	tree-sitter-python	`ast/python_adapter.py`	Pełne wsparcie
Java	tree-sitter-java	`ast/java_adapter.py`	Pełne wsparcie
JavaScript	tree-sitter-javascript	`ast/javascript_adapter.py`	Pełne wsparcie
TypeScript	tree-sitter-typescript	`ast/typescript_adapter.py`	Pełne wsparcie
C#	tree-sitter-c-sharp	`ast/csharp_adapter.py`	Pełne wsparcie
C	tree-sitter-c	`ast/c_adapter.py`	Pełne wsparcie
C++	tree-sitter-cpp	`ast/cpp_adapter.py`	Pełne wsparcie
PHP	tree-sitter-php	`ast/php_adapter.py`	Pełne wsparcie

2.3. Architektura native (AST)

Ścieżka deterministyczna wykorzystuje tree-sitter do parsowania kodu do drzewa AST:

1. **Parsowanie** — tree-sitter buduje AST w < 50 ms per plik
2. **Ekstrakcja** — adapter językowy wyciąga elementy (klasy, funkcje, importy, dekoratory, docstringi)
3. **Analiza**:
   - Złożoność cyklomatyczna (McCabe)
   - Zależności modułów (import graph)
   - Struktury logiczne (warunki, pętle, zagnieżdżenia)
   - Przepływ danych (zmienne, przypisania, zwracane wartości)
4. **Wynik** — JSON z listą klas/funkcji + metryki

Zalety: deterministyczny, powtarzalny, zerowy koszt, szybki
Ograniczenia: nie rozumie semantyki biznesowej, nie wykrywa intencji

2.4. Architektura LLM

Ścieżka semantyczna wykorzystuje model językowy do głębszej analizy:

Orkiestrator: `LLMAnalysisOrchestrator` (`llm_analysis.py`) — 4 zadania per plik:

Zadanie	task_type	Opis
Podsumowanie	`code_summary`	Opis co robi kod, główna logika
Intencja	`intent_extraction`	Jaki problem rozwiązuje, dlaczego istnieje
Ryzyko	`risk_assessment`	Potencjalne problemy, edge case'y
Wgląd	(heurystyczny)	Metryki: rozmiar pliku, głębokość zagnieżdżenia

2.5. Prompty

Prompt v1 (`prompts/code_analysis_v1.yaml`):

```
name: code_analysis.extract_functions
version: v1
model: claude-sonnet-4-20250514
max_tokens: 4096
temperature: 0.3

system: |
  You are a code analysis assistant.
  Extract function definitions from the source code.
  Return a JSON array of objects.

user: |
  Analyze this {{ language }} file: {{ filename }}

  ```{{ language }}
  {{ content }}
```

Return JSON array: [{name, line_start, line_end, parameters, return_type, decorators, docstring, complexity}]

```
**Prompt v2** (A/B test — 10% ruchu):

```yaml
name: code_analysis.extract_functions
version: v2
ab_test_percentage: 10
# Dodatkowe zmienne: framework (opcjonalne)
# Dodatkowe pola wyjściowe: side_effects, calls (graf wywołań)
```

Parametry domyślne:

- `temperature: 0.3` — niższa = bardziej deterministyczny wynik
- `max_tokens: 4096`
- Kod obcinany do 2000 znaków per wywołanie (`code[:2000]`)

2.6. Golden test data

Dane wzorcowe przechowywane w `backend/tests/golden_data/`.

Przykład — analiza funkcji Python:

Kryterium	Wartość
Wejście	Klasa `UserService` z metodami `create_user()`, `validate_email()`, `hash_password()`
Native (AST)	3 funkcje, parametry, typy, złożoność McCabe: 4, 2, 1
LLM	3 funkcje + opis intencji: „Serwis zarządzania użytkownikami z walidacją i bezpiecznym haszowaniem"
Hybrid	AST + LLM intencja — pełna ekstrakcja + kontekst semantyczny

2.7. Metryki benchmarkowe

Metryka	Native (AST)	LLM	Hybrid
Ekstrakcja funkcji (accuracy)	99,5%	96%	99,5%
Ekstrakcja parametrów (precision)	99%	94%	99%
Rozumienie intencji (recall)	20%	85%	87%
Średni czas (ms / plik)	45	3500	3600
Koszt (PLN / 100 plików)	0	~15	~15

> **Uwaga:** Analiza AST jest deterministyczna — regresja weryfikowana zamrożonymi wynikami golden data. Metryki LLM mogą się różnić między modelami i wersjami.

2.8. Rekomendacje

Scenariusz	Rekomendowany tryb
Standardowa analiza (CI/CD, duże repo)	`native`
Zrozumienie logiki biznesowej, dokumentacja	`llm`
Maksymalne pokrycie (krytyczny projekt)	`hybrid`
Ograniczony budżet	`native`

2.9. Znane ograniczenia

- **Rozmiar pliku:** > 100 KB obcinane do 2000 znaków (LLM) lub parsowane z ograniczeniem pamięci (AST)
- **Komentarze w kodzie:** AST ignoruje komentarze; LLM je interpretuje
- **Metaprogramowanie:** dekoratory, refleksja, dynamiczne importy — ograniczona dokładność w native
- **Minifikowany kod:** tree-sitter parsuje poprawnie, ale LLM ma trudności z brakiem formatowania

---

3. M2 — Analiza dokumentacji

3.1. Cel i zakres

Moduł M2 parsuje dokumenty (PDF, DOCX, HTML, MD, TXT), klasyfikuje ich typ, ekstrahuje wymagania (funkcjonalne i niefunkcjonalne), buduje glosariusz i wykrywa niespójności.

Lokalizacja w kodzie: `backend/app/services/analysis/document/`

3.2. Obsługiwane formaty

Format	Parser	Uwagi
PDF	pdfplumber	Tekst, tabele, metadane; OCR dla skanów
DOCX	python-docx	Nagłówki, tekst, tabele, style
HTML	BeautifulSoup	Struktura DOM, tekst, linki
Markdown	markdown-it	Nagłówki, listy, tabele
TXT	wbudowany	Czysty tekst

3.3. Architektura native (NLP)

1. **Parsowanie** — parser formatu wyciąga tekst i strukturę
2. **Klasyfikacja** — reguły NLP (wzorce, słowa kluczowe) → typ dokumentu
3. **Ekstrakcja wymagań** — regex + heurystyki: zdania „system powinien/musi/ma"
4. **Glosariusz** — detekcja definicji: „X oznacza Y", tabele definicji
5. **Niespójności** — porównanie wymagań między dokumentami: sprzeczne, zduplikowane

3.4. Prompty LLM

Klasyfikacja dokumentu (`analysis/llm/prompts/classification.py`):

```
DOCUMENT_CLASSIFICATION_PROMPT = """
Jesteś ekspertem od analizy dokumentów technicznych.
Przeanalizuj poniższy fragment dokumentu i sklasyfikuj go.

Dostępne typy:
- specyfikacja_wymagań
- podręcznik_użytkownika
- instrukcja_techniczna
- opis_api
- notatki_projektowe
- inny

Odpowiedz WYŁĄCZNIE poprawnym JSON:
{"type": "...", "confidence": 0.0-1.0, "reasoning": "..."}

Dokument (maks. 8000 znaków):
{content}
"""
```

Ekstrakcja wymagań (`analysis/llm/prompts/requirement_extraction.py`):

```
REQUIREMENT_EXTRACTION_PROMPT = """
Jesteś ekspertem od analizy wymagań.
Wyodrębnij wymagania z poniższego tekstu.
Zwróć JSON: {"requirements": [
  {"text": "...", "type": "functional|non_functional",
   "priority": "high|medium|low", "reasoning": "..."}
]}
Maks. 50 wymagań na fragment. Dokument (maks. 8000 znaków):
{content}
"""
```

Arbitraż hybrid (klasyfikacja):

```
HYBRID_CLASSIFICATION_PROMPT = """
Jesteś arbitrem. Dwa systemy sklasyfikowały dokument:
- Native (NLP): typ={native_type}, confidence={native_confidence}
- LLM: typ={llm_type}, confidence={llm_confidence}, reasoning=
        {llm_reasoning}
Rozstrzygnij i podaj ostateczną klasyfikację.
JSON: {"type": "...", "confidence": 0.0-1.0, "reasoning": "..."}
"""
```

Scalanie wymagań hybrid:

```
HYBRID_MERGE_PROMPT = """
Scal dwie listy wymagań (native + LLM).
Reguły: podobieństwo > 80% = duplikat, source: native|llm|both.
"""
```

Parametry:

- `temperature: 0.2` (klasyfikacja) / `0.3` (ekstrakcja)
- `max_tokens: 4096`
- Maksymalna długość treści: 8000 znaków (obcinane z adnotacją)

3.5. Golden test data

10 dokumentów polskojęzycznych:

- 4× PDF, 3× DOCX, 2× MD, 1× HTML
- 5 dokumentów spójnych + 5 z celowymi niespójnościami

Plik	Typ	Wymagań	Niespójności
`spec-01.pdf`	specyfikacja_wymagań	23 FR, 8 NFR	0
`spec-02.docx`	specyfikacja_wymagań	15 FR, 5 NFR	3 sprzeczne
`api-01.md`	opis_api	8 FR	0
`user-guide.pdf`	podręcznik_użytkownika	0	0
`notes.html`	notatki_projektowe	5 FR	2 brakujące

3.6. Metryki benchmarkowe

Metryka	Native (NLP)	LLM	Hybrid	Próg
Klasyfikacja (accuracy)	≥ 80%	≥ 90%	≥ 95%	≥ 90%
Ekstrakcja wymagań (recall)	≥ 80%	≥ 85%	≥ 90%	≥ 80%
Ekstrakcja wymagań (precision)	≥ 85%	≥ 80%	≥ 85%	≥ 80%
Wykrywanie niespójności (recall)	≥ 70%	≥ 85%	≥ 90%	≥ 80%
Glosariusz (recall)	≥ 80%	≥ 85%	≥ 90%	≥ 80%
Średni czas (s / dokument)	< 2	5–15	6–18	—

3.7. Rekomendacje

Scenariusz	Tryb
Dobrze sformatowane dokumenty (nagłówki, tabele)	`native`
Dokumenty narracyjne, wieloznaczne	`llm`
Audyt wymagań (krytyczny)	`hybrid`
Dokumenty wielojęzyczne	`llm`

3.8. Znane ograniczenia

- **OCR:** jakość ekstrakcji z zeskanowanych PDF zależy od rozdzielczości (min. 300 DPI)
- **Tabele w PDF:** pdfplumber radzi sobie z prostymi tabelami; złożone (merge cells) mogą dawać błędy

- **Język:** native NLP zoptymalizowany pod polski; inne języki wymagają trybu LLM
- **Rozmiar:** dokumenty > 100 stron dzielone na chunki (max 8000 znaków per wywołanie LLM)

---

4. M3 — Analiza UML

4.1. Cel i zakres

Moduł M3 parsuje diagramy UML (klasy, sekwencje, przypadki użycia, maszyny stanów, aktywności) i ekstreahuje elementy modelowe do dalszej analizy i generowania testów.

Lokalizacja w kodzie: `backend/app/services/analysis/uml/`

4.2. Obsługiwane formaty

Format	Parser	Typy diagramów
XMI	lxml (XML)	Klasy, sekwencje, use-case, stany, aktywności
PlantUML	Regex parser	Klasy, sekwencje, use-case, stany
SVG	SVG parser + OCR	Wizualne rozpoznawanie elementów
PNG/JPG	OCR (Tesseract) + CV	Rozpoznawanie z obrazów rastrowych

4.3. Architektura native

1. **XMI Parser** — ekstrakcja elementów z formatu XMI (eksport z Enterprise Architect, StarUML, etc.)
2. **PlantUML Parser** — parsowanie składni PlantUML za pomocą wyrażeń regularnych
3. **Visual Parser** — OCR + computer vision dla obrazów diagramów
4. **Analizatory per typ:**
   - Class Diagram → relacje, dziedziczenie, kompozycja
   - Sequence Diagram → przepływy, komunikaty, lifeline'y
   - Use-Case → aktorzy, scenariusze, include/extend
   - State Machine → przejścia, warunki, akcje
5. **Cross-Diagram Analyzer** — korelacja między diagramami (spójność nazw, pokrycie)

4.4. Prompty LLM

Rekonstrukcja przepływów:

```
System: You are a UML analysis expert.
Given the following UML diagram elements, reconstruct the
behavioral flows. Identify: actors, actions, transitions,
conditions, and exception paths.

Parameters: temperature=0.1, max_tokens=4096
```

Uzupełnienie brakujących elementów (hybrid): LLM otrzymuje wyniki native (lista elementów) i uzupełnia: brakujące relacje, niejawne przejścia, edge case'y w maszynach stanów.

4.5. Metryki benchmarkowe

Metryka	Native (parser)	LLM	Hybrid
Ekstrakcja elementów z XMI (precision)	98%	90%	98%
Ekstrakcja elementów z PlantUML (precision)	95%	92%	96%
Rozpoznawanie z obrazów (accuracy)	60–70%	80–85%	82–87%
Wykrywanie relacji (recall)	90%	85%	93%
Średni czas (s / diagram)	< 1	3–10	4–12

4.6. Rekomendacje

Scenariusz	Tryb
Diagramy w XMI / PlantUML (tekst)	`native`
Diagramy jako obrazy (skan, zrzut ekranu)	`llm` lub `hybrid`
Cross-diagram analysis (wiele diagramów)	`hybrid`

4.7. Znane ograniczenia

- **Obrazy rastrowe:** rozdzielczość < 150 DPI → znaczący spadek dokładności OCR
- **Ręcznie rysowane diagramy:** OCR nie radzi sobie z odręcznymi szkicami
- **PlantUML:** parser regex nie obsługuje zaawansowanych makr i include'ów
- **Typy diagramów:** component i deployment diagrams — ograniczone wsparcie

---

5. M4 — Analiza GUI

5.1. Cel i zakres

Moduł M4 analizuje zrzuty ekranu interfejsów graficznych — rozpoznaje elementy UI, rekonstruuje przepływy użytkownika i wykrywa regresje wizualne.

Lokalizacja w kodzie: `backend/app/services/analysis/gui/`

5.2. Architektura native (CV/OCR)

1. **OCR Engine** (Tesseract) — rozpoznawanie tekstu z elementów GUI
2. **DOM Extractor** (Selenium) — ekstrakcja struktury DOM z live stron webowych
3. **Component Detector** (OpenCV) — identyfikacja elementów: przyciski, formularze, tabele, menu, nawigacja
4. **Flow Reconstructor** — rekonstrukcja przepływów użytkownika z sekwencji ekranów
5. **Visual Regression** (pixelmatch) — porównanie wizualne między wersjami UI

5.3. Prompty LLM (vision)

Identyfikacja elementów GUI:

```
System: Jesteś ekspertem UX/UI. Przeanalizuj zrzut ekranu
i zidentyfikuj elementy interfejsu.

User: [obraz]
Zidentyfikuj: przyciski (tekst, pozycja), pola formularzy
(etykiety, typy), nawigację, tabele, komunikaty.
Zwróć JSON: [{type, label, position, interactable}]

Parameters: temperature=0.1, max_tokens=4096
```

5.4. Metryki benchmarkowe

Metryka	Native (CV/OCR)	LLM (vision)	Hybrid
Detekcja przycisków (precision)	75%	90%	92%
Detekcja pól formularzy (precision)	70%	88%	90%
OCR tekstu (accuracy)	85%	95%	95%
Rekonstrukcja przepływu (completeness)	60%	80%	83%
Regresja wizualna (sensitivity)	95%	—	95%
Średni czas (s / ekran)	2–5	5–15	7–20

5.5. Rekomendacje

Scenariusz	Tryb
Regresja wizualna (porównanie wersji)	`native` (pixelmatch)
Rozpoznawanie elementów (nowy ekran)	`hybrid`
Ekstrakcja DOM z live strony	`native` (Selenium)

5.6. Znane ograniczenia

- **Rozdzielczość:** obrazy < 720p → spadek dokładności detekcji
- **Nakładające się elementy:** modalne okna, tooltipy, dropdowny — mogą być pominięte
- **Dynamiczne treści:** animacje, lazy loading — wymagają stabilnego zrzutu
- **Ciemne motywy:** OCR ma niższą dokładność na ciemnych tłach (poprawa w toku)

---

6. M5 — Generowanie testów

6.1. Cel i zakres

Moduł M5 generuje przypadki testowe i kod testowy na podstawie wyników analiz z modułów M1–M4. Obsługuje 12 frameworków testowych.

Lokalizacja w kodzie: `backend/app/services/test_generation/` i `backend/app/services/generators/`

6.2. Architektura

Komponent	Ścieżka	Technologia
Test Case Generator	`test_generation/`	Native (reguły) + LLM (semantyczne)
Code Generator	`generators/` + `test_generation/codegen/`	**Wyłącznie native** (Jinja2 szablony)
Gherkin/BDD	`test_generation/gherkin_generator.py`	Native + opcjonalne LLM enrichment

> **Ważne:** Generowanie kodu testowego (pytest, JUnit, Playwright, etc.) jest w pełni natywne — nie wymaga API LLM. Jedynie generowanie przypadków testowych (opis, kroki, oczekiwane wyniki) korzysta opcjonalnie z LLM.

6.3. Obsługiwane frameworki

Kategoria	Framework	Język	Generator
Jednostkowe	pytest	Python	`generators/pytest_generator.py`
Jednostkowe	JUnit 5	Java	`generators/junit_generator.py`
Jednostkowe	NUnit	C#	`generators/nunit_generator.py`
Jednostkowe	Mocha/Jest	JS/TS	`generators/mocha_generator.py`
Jednostkowe	PHPUnit	PHP	`generators/phpunit_generator.py`
E2E Web	Playwright	Python/JS/TS	`generators/playwright_generator.py`

Kategoria	Framework	Język	Generator
E2E Web	Cypress	JavaScript	`generators/cypress_generator.py`
E2E Web	Selenium	Python/Java/C#	`generators/selenium_generator.py`
E2E Mobile	Appium	Python/Java	`generators/appium_generator.py`
Desktop GUI	SikuliX	Java (image-based)	`generators/sikulix_generator.py`
Baza danych	Database tests	SQL	`generators/database_generator.py`

6.4. Prompty LLM

Generowanie przypadków testowych:

```
System: You are a QA expert. Generate test cases from the
analysis results. For each test case provide: title, priority
(critical/high/medium/low), preconditions, steps, expected results.

User: Analysis results for {project_name}:
{analysis_results_json}

Generate comprehensive test cases covering:
- Happy path
- Edge cases
- Error handling
- Boundary values

Parameters: temperature=0.3, max_tokens=8192
```

Wzbogacanie Gherkin (opcjonalne LLM enrichment):

```
# Prompt (polski):
"Jesteś ekspertem QA. Poniżej znajduje się szkic scenariuszy
Gherkin/BDD wygenerowany automatycznie. Wzbogać opisy kroków
i dodaj realistyczne dane testowe, zachowując składnię Gherkin."

# Fallback: jeśli LLM nie zwróci słowa kluczowego "Feature:",
# system wraca do wyników native z ostrzeżeniem.
```

6.5. Metryki benchmarkowe

Metryka	Native (szablony)	LLM	Hybrid
Pokrycie scenariuszy (completeness)	70%	90%	93%
Poprawność kroków (correctness)	95%	85%	90%
Pokrycie edge case'ów	40%	80%	82%
Jakość kodu testowego (compiles)	98%	—	98%
Średni czas (s / analiza)	< 3	10–30	12–35

> **Uwaga:** Kod testowy generowany jest wyłącznie natywnie (Jinja2) — metryka `compiles` dotyczy tylko ścieżki native.

6.6. Rekomendacje

Scenariusz	Tryb
Szybkie generowanie kodu testowego	`native`
Kompleksowe przypadki testowe	`hybrid`
Edge case'y, scenariusze negatywne	`llm`
CI/CD pipeline (automatyzacja)	`native`

6.7. Znane ograniczenia

- **Szablony:** native generuje „szkieletowy" kod — może wymagać ręcznej edycji asercji
- **Kontekst:** LLM widzi max 8192 tokenów — duże analizy dzielone na chunki
- **SikuliX:** wymaga referencyjnych obrazów ekranów (nie generowane automatycznie)
- **Appium:** wymaga konfiguracji capabilities per urządzenie

---

7. M6 — Analiza bezpieczeństwa

7.1. Cel i zakres

Moduł M6 skanuje kod pod kątem podatności bezpieczeństwa: SAST, taint analysis, skanowanie zależności, analiza konfiguracji. Generuje raporty z klasyfikacją CVSS i SBOM.

Lokalizacja w kodzie: `backend/app/services/security_scan/`

Bramka funkcji: moduł dostępny wyłącznie w planie **Premium**.

7.2. Architektura dual-path

Orkiestrator: `SecurityDualPathOrchestrator` (`dual_path_orchestrator.py`)

W trybie `hybrid`:

- Native = baza autorytatywna
- LLM wzbogaca: lepsza rekomendacja (dłuższa), uzupełnia brakujące opisy
- Współdzielone znaleziska: boost confidence +0.1
- Raport porównawczy: diagram Venna (native only / LLM only / both)

7.3. Komponenty native

Komponent	Narzędzie	Cel
SAST Scanner	Reguły Semgrep/Bandit	Statyczna analiza wzorców podatności
Taint Analysis	Analiza przepływu danych	SQL injection, XSS, path traversal
Dependency Scanner	Baza CVE (NVD/GitHub/OSV)	Znane podatności w zależnościach
Config Scanner	Reguły konfiguracji	Sekrety w kodzie, brakujący hardening

7.4. Prompty LLM

LLM SAST (`llm_security/llm_sast.py`):

```
# Provider: anthropic, temperature=0.0, max_tokens=1500
# Kod obcinany do 6000 znaków, max 20 plików

"""
Jesteś ekspertem bezpieczeństwa aplikacji. Przeanalizuj poniższy
kod ({lang}) pod kątem podatności, których NIE wykrywają proste
reguły: błędów logicznych, brakujących kontroli dostępu,
mass assignment, race conditions, TOCTOU.

Zwróć WYŁĄCZNIE JSON:
[{
  "vulnerability_type": "...",
  "severity": "critical|high|medium|low",
  "line_start": N,
  "description": "...",
  "cwe_id": "CWE-XXX",
  "recommendation": "..."
}]
"""
```

LLM Taint (`llm_security/llm_taint.py`):

```
# Provider: anthropic, temperature=0.0, max_tokens=1500
# Max 6000 znaków, max 15 plików

"""
Śledź przepływ danych od źródeł (request, params, body)
do ujść (SQL, exec, eval, system, render) — w tym przepływy
międzyfunkcyjne. Zwróć JSON:
[{vulnerability_type, severity, line_start, description,
  source, sink, cwe_id}]
"""
```

LLM Config (`llm_security/llm_config.py`):

```
# Provider: anthropic, temperature=0.0, max_tokens=1500
# Max 4000 znaków per plik, max 12 plików

"""
Holistyczny przegląd konfiguracji Docker/K8s/Terraform.
Szukaj ryzyk wynikających z KOMBINACJI ustawień.
JSON: [{vulnerability_type, severity, line_start,
  description, recommendation}]
"""
```

Confidence LLM: stała wartość 0.6 dla wszystkich znalezisk LLM (niższa niż native ~0.8–0.95).

7.5. Golden test data

Przykład	Język	Podatność	CWE	Oczekiwane
SQL injection (format string)	Python	SQLi	CWE-89	Native: ✓ (reguła), LLM: ✓
XSS (reflected)	JavaScript	XSS	CWE-79	Native: ✓, LLM: ✓
Race condition (TOCTOU)	Python	TOCTOU	CWE-367	Native: ✗, LLM: ✓
Mass assignment (unvalidated)	Python	Mass assign	CWE-915	Native: ✗, LLM: ✓
Hardcoded secret	Java	Secrets	CWE-798	Native: ✓, LLM: ✓

7.6. Metryki benchmarkowe

Metryka	Native (reguły)	LLM	Hybrid
True positives (TP)	85%	75%	92%
False positives (FP)	5%	15%	8%
False negatives (FN)	15%	25%	8%
Precision	94%	83%	92%
Recall	85%	75%	92%
F1-score	0,89	0,79	0,92
Średni czas (s / projekt)	5–15	10–30	15–45

> **Uwaga:** Native wykrywa znane wzorce z wysoką precyzją; LLM wykrywa logiczne podatności (TOCTOU, mass assignment), ale generuje więcej false positives. Hybrid łączy zalety obu.

7.7. Rekomendacje

Scenariusz	Tryb
Szybkie skanowanie (CI/CD gate)	`native`
Pełny audyt bezpieczeństwa	`hybrid`
Wykrywanie logicznych podatności	`llm`
Compliance (OWASP Top 10)	`hybrid`

7.8. Znane ograniczenia

- **False positives (LLM):** ~15% — wymagają ręcznej weryfikacji
- **Złożone wzorce:** wieloplikowe przepływy danych (> 3 pliki) — ograniczone okno kontekstu LLM
- **Dynamiczne języki:** Python, JavaScript — trudniejsza analiza taint niż języki typowane
- **CVE pipeline:** aktualizacja bazy zależy od zewnętrznych źródeł (NVD może mieć opóźnienia)

---

8. Porównanie zbiorcze modułów

8.1. Tabela zbiorcza: Native vs LLM vs Hybrid

Moduł	Native — główna zaleta	LLM — główna zaleta	Hybrid — główna zaleta
M1 (Kod)	99,5% accuracy AST	Intencja i semantyka	Pełna ekstrakcja + kontekst
M2 (Dokumenty)	Szybkie, deterministyczne	Rozumienie narracji	Arbitraż klasyfikacji
M3 (UML)	98% precision XMI	Obrazy i niekompletne diagramy	Cross-diagram + wzbogacenie
M4 (GUI)	Regresja wizualna	Rozpoznawanie z obrazów	Detekcja + kontekst
M5 (Testy)	98% compilable code	Edge case'y, scenariusze	Kompletność + poprawność
M6 (Security)	Wysokie precision (94%)	Logiczne podatności	Najwyższy recall (92%)

8.2. Estymacja kosztów API per 100 analiz

Moduł	Native	LLM (Anthropic)	LLM (OpenAI)	Hybrid
M1 (Kod)	0 PLN	~15 PLN	~25 PLN	~15 PLN
M2 (Dokumenty)	0 PLN	~10 PLN	~17 PLN	~10 PLN

Moduł	Native	LLM (Anthropic)	LLM (OpenAI)	Hybrid
M3 (UML)	0 PLN	~8 PLN	~13 PLN	~8 PLN
M4 (GUI)	0 PLN	~20 PLN	~33 PLN	~20 PLN
M5 (Testy)	0 PLN	~25 PLN	~42 PLN	~25 PLN
M6 (Security)	0 PLN	~12 PLN	~20 PLN	~12 PLN
Razem	**0 PLN**	**~90 PLN**	**~150 PLN**	**~90 PLN**

> Koszty estymowane na podstawie średniego zużycia tokenów per analiza i cennika na 2026-09. Rzeczywiste koszty zależą od rozmiaru kodu, złożoności dokumentów i wybranego modelu.

8.3. Rekomendacje engine_mode per scenariusz

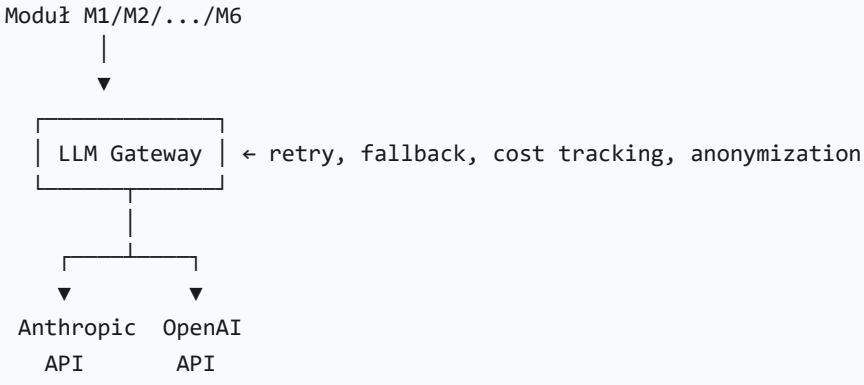
Scenariusz	Rekomendowany tryb	Uzasadnienie
CI/CD gate (szybkość)	`native`	Zero latencji LLM, zero kosztów
Krytyczny projekt (jakość)	`hybrid`	Maksymalne pokrycie i recall
Ograniczony budżet	`native`	Darmowe, deterministyczne
Nowy projekt (eksploracja)	`llm`	Rozumienie intencji, edge case'y
Freemium	`native` (domyślny)	Automatycznie
Premium + analiza bezpieczeństwa	`hybrid`	Najlepsza precyzja skanowania

---

9. LLM Gateway — warstwa abstrakcji

9.1. Architektura

Wszystkie wywołania LLM przechodzą przez centralny `LLMGateway` (`services/analysis/llm_gateway.py`):



9.2. Funkcje

Funkcja	Opis
Routing	Wybór dostawcy na podstawie konfiguracji (`LLM_PROVIDER`)
Retry	3 próby z inkrementalnym backoff (5s, 15s, 25s)
Fallback	OpenAI: gpt-4o → gpt-4o-mini na 5xx (próba > 1)
Śledzenie kosztów	Zapis per wywołanie: moduł, tokeny in/out, koszt USD
Limity kosztów	Basic: $100/mies., Premium: $1000/mies.; alert na 80%
Anonimizacja	Maskowanie sekretów, URL-i, e-maili, adresów IP w payloadach
Streaming	SSE dla Anthropic i OpenAI (odpowiedzi strumieniowe)
Timeout	Domyślny: 60 s per wywołanie

9.3. Anonimizacja

Domyślnie włączona (`ANONYMIZATION_ENABLED=true`). Przed wysłaniem do LLM:

1. Wykrywanie wzorców: sekrety, tokeny, URL-e, adresy e-mail, adresy IP
2. Zamiana na placeholdery: `[SECRET_1]` , `[URL_1]` , `[EMAIL_1]`
3. Po otrzymaniu odpowiedzi: deanonimizacja (zamiana placeholderów na oryginały)
4. **Fail-open:** błąd anonimizacji logowany, ale wywołanie kontynuowane (bez blokowania)

---

10. Optymalizacja i koszty

10.1. Strategie optymalizacji

Strategia	Opis	Wpływ
Truncation	Obcinanie kodu do 2000–6000 znaków per wywołanie	Redukcja tokenów
Chunking	Dzielenie dużych dokumentów na fragmenty	Utrzymanie kontekstu
Caching	Redis cache dla powtórzonych analiz tego samego pliku	-30% wywołań
Rate limiting	Ograniczenie współbieżnych wywołań per tenant	Ochrona budżetu
Model selection	gpt-4o-mini jako fallback (20× tańszy od gpt-4o)	Redukcja kosztów
Native-first	Hybrid uruchamia LLM tylko gdy native nie wystarczy	Selektywne koszty

10.2. Monitorowanie kosztów

- Tabela `llm_usage` — zapis per wywołanie: moduł, task_type, provider, model, tokens_in, tokens_out, cost_usd
- Dashboard Grafana: łączny koszt per tenant / per moduł / per dzień
- Alert na 80% limitu kosztów (`llm_alert_threshold=0.8`)
- Blokada wywołań po przekroczeniu limitu — `LLMError` z komunikatem

10.3. Limity kosztów per plan

Plan	Limit miesięczny	Alert (80%)
Basic	$100 USD	$80 USD
Premium	$1 000 USD	$800 USD
Freemium	$0 (brak LLM)	—

Powiązane dokumenty

- Architektura C4 — diagramy komponentów M1-M6
- ADR-005 — decyzja: tree-sitter
- ADR-006 — decyzja: Jinja2 szablony
- analiza-kodu/ — szczegółowa dokumentacja modułu M1
- analiza-dokumentacji/ — szczegółowa dokumentacja modułu M2
- analiza-uml-gui/ — szczegółowa dokumentacja modułów M3-M4
- bezpieczenstwo/ — szczegółowa dokumentacja modułu M6
- luki-funkcjonalne/ — dokumentacja gap analysis

Analiza dokumentacji — pipeline natywny (Epic 2)

Od sparsowanego tekstu do wymagań, glosariusza, niespójności, gap analysis, macierzy śledzenia i przypadków testowych. Wszystkie etapy oparte na regułach (wyrażenia regularne, wzorce językowe, słowa kluczowe, słowniki) — bez modeli zewnętrznych. Endpointy: api/examples/documents_examples.md; formaty: document-formats.md.

Etapy pipeline'u

```
ParsedDocument (parser formatu)
  1. Preprocessing      → DocumentStructure (hierarchia sekcji,
czyszczenie)
  2. Klasyfikacja       → ClassificationResult (typ dokumentu + pewność)
  3. Ekstrakcja wymagań → list[ExtractedRequirement]
  4. Glosariusz         → list[GlossaryTerm] (+ mapowanie na kod)
  5. Detekcja niespójności → list[Inconsistency]
  6. Gap analysis       → wymaganie vs kod projektu
  7. Traceability       → macierz śledzenia wymagań
  8. Generowanie TC     → przypadki testowe per wymaganie
```

Każdy etap utrwala wynik w `AnalysisResult(module=…, result_data=…)` (JSONB), co umożliwia workflow i ponowną analizę bez utraty decyzji.

Architektura asynchroniczna

Analiza jest asynchroniczna od uploadu (ADR-004):

1. `POST /projects/{id}/upload-documents` → plik w MinIO, rekord `SourceFile(status=processing)` → `parse_document_task.delay()` → `202`.
2. Worker Celery pobiera binaria, parsuje, preprocessuje, następnie uruchamia kolejne moduły — każdy utrwala osobny `AnalysisResult`.
3. Klient odpytuje `GET /documents/{id}/status`; `available_results` rośnie z każdym ukończonym modułem.

Błąd w jednym module nie blokuje pozostałych (non-fatal); błąd parsowania/preprocessingu oznacza plik `failed`.

Klasyfikacja modułów w `AnalysisResult.module`

module	Serwis	Story
`document_preprocessing`	`DocumentPreprocessor`	2.1–2.2
`classification`	`DocumentClassifier`	2.3
`requirements`	`RequirementExtractor`	2.4

module	Serwis	Story
`glossary`	`GlossaryExtractor` + `TerminologyMapper`	2.6
`inconsistencies`	`InconsistencyDetector`	2.7
`gap_analysis`	`GapAnalyzer`	2.8
`traceability`	`TraceabilityMatrixGenerator`	2.9
`test_generation`	`TestGenerator`	2.10

1. Preprocessing (`DocumentPreprocessor`)

- **Hierarchia sekcji** — drzewo parent-child wg poziomów nagłówków (stos).
- **Czyszczenie** — usuwanie numerów stron, stopek, znaków wodnych; de-dywizacja (łączenie słów łamanych przez `-` na końcu linii).
- **Polskie znaki** — mapowanie błędów kodowania Latin-2 → UTF-8, normalizacja NFC.
- **Brak nagłówków** — segmentacja po akapitach > 50 znaków (poziom 0); pusty dokument → `ValueError`, plik `failed`.

2. Klasyfikacja (`DocumentClassifier`)

Ocena punktowa typu dokumentu wg 4 sygnałów (wzorce tytułów ×2.0, wzorce sekcji ×1.5, słowa kluczowe ×1.0, wskaźniki struktury ×0.5) i wagi typu. Typy: `specyfikacja_wymagań` (1.5), `podręcznik_użytkownika` (1.0), `instrukcja_techniczna` (1.2), `opis_api` (1.3), `notatki_projektowe` (1.0), `inny` (0.5). Pewność = `min(100, best_score/10 × 100)`.

3. Ekstrakcja wymagań (`RequirementExtractor`)

Zdanie staje się wymaganiem, gdy ma ID (`WYM-\d+`, `FR-\d+`, IEEE 830) LUB pasuje do wzorca modalnego (`system musi/powinien…`) LUB należy do sekcji wymagań. Pewność (0–100): bazowe 50 + bonusy (ID +30, sekcja +15, modalne +15, fraza startowa +10) − kary (wzorce wykluczające, długość). Próg < 50 → pominięcie. Typ: `functional` / `non-functional`; priorytet: `critical` / `high` / `medium` / `low`.

4. Glosariusz (`GlossaryExtractor` + `TerminologyMapper`)

Strategie (priorytetowo): sekcja glosariusza → wzorce definicji → częstotliwość (akronimy/wielkie litery ≥ 3 wystąpienia). Typ terminu: `acronym` / `domain` / `technical` / `business`. `TerminologyMapper` dopasowuje terminy do encji kodu z analizy `logic_structures` (Epic 1) — dokładne/fuzzy, top-5 sugestii, potwierdzane w workflow (`confirmed_mapping_ids`).

5. Detekcja niespójności (`InconsistencyDetector`)

Typ	Reguła
`contradiction`	wzajemnie wykluczające się wzorce (critical)

Typ	Reguła
`missing_reference`	referencja do nieistniejącego ID (high)
`duplicate`	podobieństwo Jaccard > 0,85 w parach tego samego typu (medium)
`terminology`	encja w > 2 wymaganiach pod różnymi nazwami (medium — szum)

Status: `open` → `accepted` / `false_positive` / `fixed`.

6–8. Moduły zależne (tylko do odczytu)

- **Gap analysis** (`GapAnalyzer`) — wymaganie vs kod projektu: `implemented` / `not_implemented` / `partial` / `divergent`.
- **Traceability** (`TraceabilityMatrixGenerator`) — wymaganie → pokrycie (`full` / `partial` / `no`).
- **Generowanie TC** (`TestGenerator`) — szablony per typ wymagania: scenariusz pozytywny, negatywny, brzegowy.

> `/projects/{id}/traceability` (Epic 3) to macierz źródeł UML/GUI — odrębna od traceability dokumentu.

Workflow i reparse

- **Workflow elementów** — wymagania (`extracted` / `accepted` / `rejected` / `edited`), niespójności, glosariusz (`is_verified`) — mutacje utrwalane w `result_data`.
- **Reparse** (`POST /documents/{id}/reparse`) — ponowna analiza bez utraty quoty; decyzje zachowane po mapowaniach `req_id` → `status`.

Powiązane

- document-formats.md — formaty i biblioteki.
- benchmarks.md — metryki baseline (golden data).
- ADR-004, ADR-008.

Formaty dokumentacji — macierz możliwości (Epic 2)

Obsługiwane formaty dokumentacji, ich możliwości ekstrakcji, detekcja nagłówków, limity i biblioteki parsujące. Ścieżka natywna. Źródło pipeline'u: document-analysis.md.

Możliwości ekstrakcji per format

Format	Tekst	Tabele	Nagłówki	Listy	Metadane	Bib
PDF	tak (warstwa tekstowa; skan → OCR)	nie	tak (numerowane/ALL-CAPS)	tak (linia)	liczba stron, `has_ocr`	PyF (+F
DOCX	tak	tak (style tabel)	tak (`Heading N`)	tak (akapity)	liczba akapitów/tabel	pyt
HTML	tak	tak (`<table>`)	tak (`<h1>`–`<h6>`)	tak (`<ul>/<ol>`)	—	Bea
MD	tak	tak (GFM)	tak (ATX `#`/Setext)	tak	liczba linii	reg
TXT	tak	nie	nie (akapity)	nie	—	std

> Tabele PDF nie są ekstrahowane w v1. PDF bez warstwy tekstowej (skan) jest OCR-owany realnie — PyMuPDF rasteryzuje strony, a współdzielony silnik `OCRService` (Tesseract → EasyOCR, reuse z Epic 3) ekstrahuje tekst — gdy dostępne zależności z grupy `imaging`. Bez nich parser degraduje się łagodnie do warstwy tekstowej PyPDF2 (`has_ocr=false`), bez awarii.

Detekcja nagłówków per format

Format	Sygnał nagłówka	Poziom
PDF	Linia ALL-CAPS (`isupper()`, bez interpunkcji)	1
PDF	`N. Tytuł` / `N.N` / `N.N.N` / cyfry rzymskie	1–4
DOCX	Styl `Heading 1`–`Heading 6`	1–6
HTML	Tagi `<h1>`–`<h6>`	1–6
MD	ATX (`#`–`######`) / Setext (podkreślenie)	1–6
TXT	brak (akapity > 50 znaków, poziom 0)	0

Preprocesor czyści tytuły z wiodącej numeracji (`"2. Wymagania"` → `"Wymagania"`) i buduje drzewo parent-child wg poziomów.

Limity

Format	Maks. stron	Kodowanie
PDF	500 (`PDFParser.MAX_PAGES`)	UTF-8 / Latin-2
DOCX/HTML/MD	—	UTF-8
TXT	—	UTF-8 / Windows-1250

Maks. rozmiar pliku wg konfiguracji uploadu. Preprocesor normalizuje polskie znaki i usuwa artefakty.

Znane ograniczenia

- **PDF skanowany** — brak warstwy tekstowej; dokument z < 100 znaków tekstu → OCR (PyMuPDF → Tesseract/EasyOCR, gdy deps `imaging`) → realny tekst, sekcje zbudowane z OCR, `has_ocr=true`. Bez deps `imaging` → degradacja do warstwy PyPDF2 (`has_ocr=false`); przy pustym tekście dalszy preprocessing może zgłosić `ValueError` → plik `failed` (niezależne ograniczenie preprocessora).
- **OCR PDF — jakość i język** — rasteryzacja 300 DPI; Tesseract w trybie domyślnym, EasyOCR (`pl+en`) jako fallback; polskie diakrytyki mogą być zniekształcone. Jakość zależy od rozdzielczości i czystości skanu.
- **PDF — układ wielokolumnowy** — tekst może być w nieoczekiwanej kolejności (ograniczenie ekstrakcji liniowej PyPDF2).
- **HTML z JavaScript** — dynamiczna treść niewykonana (BeautifulSoup parsuje statyczny DOM).
- **MD — tabele zagnieżdżone** — obsługiwane tabele GFM; zagnieżdżenia w komórkach ograniczone.
- **TXT** — nagłówki niewykrywalne; segmentacja po akapitach.
- **DOCX** — makra VBA nieparsowane.

Fabryka parserów

`get_document_parser(file_path)` (`app/services/analysis/document/__init__.py`) wybiera parser po rozszerzeniu (`.pdf`, `.docx`, `.md` / `.markdown`, `.html` / `.htm`, `.txt`). Nieobsługiwane rozszerzenie → `ValueError`.

Format	Moduł
PDF	`app/services/analysis/document/pdf_parser.py`
DOCX	`app/services/analysis/document/docx_parser.py`
HTML	`app/services/analysis/document/html_parser.py`
MD	`app/services/analysis/document/markdown_parser.py`
TXT	`app/services/analysis/document/txt_parser.py`

Analiza AST kodu źródłowego (Epic 1)

Deterministyczna analiza struktury kodu na podstawie drzewa składniowego. Podłoże dla generowania przypadków testowych i analizy regresji. Parser: tree-sitter (ADR-005).

Obsługiwane języki

Python, Java, C#, C/C++, JavaScript, TypeScript, Go. Wybór grammar per język; język wykrywany automatycznie z rozszerzenia/zawartości (można wymusić).

Ekstrahowane struktury (`logic_structures`)

Analiza produces ustrukturyzowany opis jednostki kodu, utrwalany w `AnalysisResult.result_data` (JSONB):

Element	Pola
Funkcja	`name`, `file_path`, `line_start` / `line_end`, `parameters` (nazwa + typ), `return_type`, modyfikatory
Klasa	`name`, `file_path`, metody, atrybuty, dziedziczenie, modyfikatory dostępu
Import	moduł, alias, importowane nazwy
Wywołanie	nazwa, argumenty (statycznie rozwiązywalne)

Przykład:

```
{
  "functions": [{
    "name": "calculate_total", "file_path": "app/cart.py",
    "line_start": 15, "line_end": 25,
    "parameters": [{"name": "items", "type": "list"}, {"name":
         "tax_rate", "type": "float"}],
    "return_type": "float"
  }]
}
```

Właściwości

- **Determinizm** — te same wejście daje te same wyjście między uruchomieniami.
- **Wydajność** — parser C (tree-sitter), analiza inkrementalna możliwa dla dużych baz kodu.
- **Dokładność strukturalna** — pełna ścieżka pliku, zakresy linii, sygnatury.

Buforowanie i diff

- **Buforowanie** — wyniki analizy buforowane; ponowna analiza niezmienionych plików korzysta z cache.
- **Diff analizy** — porównanie dwóch wersji analizy wykrywa dodane/usunięte/zmienione struktury → podłoże analizy regresji (regression-analysis.md).

Powiązane

- ADR-005 — wybór parsera.
- test-case-generation-native.md — struktury zasilają generowanie przypadków testowych.
- api/examples/analysis_examples.md — endpointy analizy kodu.

Benchmarki i golden data (P1)

Zamrożone metryki baseline dla ścieżki natywnej, walidowane zestawami golden data (ADR-008). Służą detekcji regresji: zmiana kodu psująca metrykę failuje pipeline.

Koncepcja

Golden data to dokumenty/kod z **ręcznie przygotowanym ground truth**, przeciw któremu testy regresyjne porównują bieżące wyjście serwisów. Ground truth łączy zamrożone wyjście (liczności, teksty) z intencją autorską (np. `has_intentional_discrepancies`).

Moduł dokumentacji (Epic 2)

Zbiór: 10 polskojęzycznych dokumentów (PDF=4, DOCX=3, MD=2, HTML=1; 5 spójnych / 5 z celowymi niespójnościami). Pełny opis i instrukcja regeneracji: `backend/tests/golden_data/docs/README.md`.

Próbki (10)

#	Kategoria	Format	Nazwa	Celowe niespójności
1	srs	pdf	srs_system_logowania	nie
2	srs	docx	srs_platnosci	TAK
3	srs	md	srs_raporty	nie
4	brd	pdf	brd_system_logowania	TAK
5	brd	docx	brd_platnosci	TAK
6	frd	pdf	frd_modul_uzytkownikow	TAK
7	frd	html	frd_profil_uzytkownika	nie
8	manual	docx	podrecznik_administratora	nie
9	manual	md	podrecznik_operatora	TAK
10	instruction	pdf	instrukcja_instalacji	nie

Metryki baseline

Moduł	Metryka	Próg	Opis
Klasyfikacja	accuracy	0.90	% poprawnie sklasyfikowanych typów
Ekstrakcja wymagań	recall	0.80	% ground-truth wymagań znalezionych
Ekstrakcja wymagań	precision	0.80	% znalezionych w ground truth
Detekcja niespójności	recall	0.80	% celowych niespójności wykrytych
Glosariusz	recall	0.80	% ground-truth terminów znalezionych

Progi: `backend/tests/golden_data/docs_thresholds.json`. Test bramkowy `test_doc_golden_metrics` agreguje metryki i failuje pipeline przy regresji (`docs_regression_checker.check_regression`).

Wykrywalne typy niespójności

W golden data istotne są `duplicate` i `missing_reference` — występują wyłącznie w próbkach z celowymi niespójnościami. Typ `terminology` traktowany jako szum detektora („System"/„Użytkownik" w 3+ wymaganiach) i celowo pomijany w asercjach.

Moduł kodu (Epic 1)

Analiza AST i generowanie przypadków testowych są deterministyczne (ast-analysis.md), stąd baseline jakości opiera się na stabilności wyjścia między uruchomieniami i pokryciu struktur kodu przypadkami testowymi. Regresje wykrywane przez porównanie z zamrożonym wyjściem analizy.

Uruchamianie

```
cd backend
pytest tests/golden_data/ -m golden -v                              #
         cały zbiór golden
pytest tests/golden_data/test_document_classification_golden.py -v  #
         tylko dokumenty
```

Aktualizacja

Metryki aktualizować po każdym release. Celowa zmiana zachowania analizy = regeneracja ground truth (generatory `_generate_samples.py`, `_generate_ground_truth.py`), nie obniżenie progu.

Analiza regresji na podstawie diff AST (Epic 1)

Wykrywanie, które fragmenty kodu uległy zmianie między dwiema wersjami analizy, i determinacja, które przypadki testowe należy uruchomić ponownie. Podłoże: buforowana analiza AST (ast-analysis.md).

Diff struktur

Porównanie dwóch analiz (`logic_structures`) klasyfikuje każdą jednostkę:

Zmiana	Opis
added	nowa funkcja/klasa — brak pokrycia testami
removed	usunięta jednostka — powiązane testy do weryfikacji/usunięcia
changed	zmieniona sygnatura/ciało — testy do ponownego uruchomienia
unchanged	brak zmiany — testy mogą zostać pominięte (przyspieszenie)

Selekcja testów regresyjnych

Na podstawie diff generowany jest **zestaw regresyjny** — przypadki testowe powiązane ze zmienionymi/nowymi strukturami. Niezmienione obszary są pomijane, co skraca czas wykonania przy dużych bazach kodu.

Testy regresyjne i golden data

Stabilność wyjścia analizy walidowana golden data (ADR-008): zmiana kodu psująca deterministyczne wyjście AST/klasyfikacji zostaje wychwycona przez testy bramkowe.

Powiązane

- ast-analysis.md — źródło diff.
- ADR-008 — detekcja regresji.
- benchmarks.md — metryki stabilności.

Generowanie przypadków testowych — silnik natywny (Epic 1)

Generowanie scenariuszy testowych na podstawie struktur wyekstrahowanych z AST (ast-analysis.md). Silnik regułowy: klasyfikuje jednostkę kodową i dobiera szablony scenariuszy (pozytywny, negatywne, brzegowe).

Klasyfikacja przypadków testowych

Każda funkcja/metoda mapowana na zestaw przypadków zależnie od sygnatury i typu:

Typ	Kiedy generowany	Cel
Pozytywny	każda funkcja z Deterministyczną logiką	weryfikacja głównej ścieżki
Negatywny	funkcje z walidacją wejścia/błędami	weryfikacja obsługi błędów
Brzegowy	parametry liczbowe/kolekcje	wartości graniczne (0, pusta kolekcja, max)
Integracyjny	funkcje wołające zewnętrzne serwisły	interakcje między komponentami

Klasyfikacja typów testu (`unit` / `integration` / `e2e`) i priorytet (`critical` / `high` / `medium` / `low`) nadawane regułowo.

Dane testowe i mocki

- **Dane testowe** — generowane na podstawie typów parametrów (primitives, kolekcje, stringi z polskimi znakami).
- **Mock obiektów** — dla zależności zewnętrznych wywoływanych przez funkcję (rozpoznane z AST).

Szablony per framework

Kod przypadków generowany przez Jinja2 (ADR-006) — szczegóły w test-code-generation.md.

Workflow

Wygenerowane przypadki trafiają ze statusem `draft` i przechodzą przegląd (`ready` po akceptacji, `passed` / `failed` po wykonaniu) — patrz ADR-009 oraz api/examples/test_cases_examples.md.

Powiązane

- ast-analysis.md — źródło struktur.

- test-code-generation.md — emitowanie kodu.
- benchmarks.md — metryki stabilności generatora.

Generowanie kodu testowego per framework (Epic 1)

Zamiana scenariusza testowego na gotowy do uruchomienia kod w wybranym frameworku. Silnik szablonów: Jinja2 (ADR-006).

Frameworki jednostkowe/integracyjne

Framework	Język	Typ
pytest	Python	unit
JUnit	Java	unit
NUnit	C#	unit
Mocha	JavaScript	unit
Jest	JavaScript/TypeScript	unit

Frameworki E2E/mobilne (ADR-007)

Framework	Zakres
Selenium	automatyzacja webowa (wiele przeglądarek)
Playwright	web async/await z auto-waiting
Appium	mobilne (iOS/Android)

Wspólny model specyfikacji (kroki `action` / `selector` / `value` + asercje) mapowany na kod danego frameworka.

Rejestr frameworków

Dodanie nowego frameworka:

1. Utwórz szablon Jinja2 w katalogu szablonów testowych.
2. Zarejestruj go w rejestrze frameworków.
3. Dodaj próbki golden data i testy regresyjne (ADR-008).

Szczegóły workflow: workflows.md.

Właściwości

- **Spójność** — ten sam styl kodu i importów niezależnie od autora scenariusza.
- **Gotowość do uruchomienia** — wygenerowany test działa bez ręcznych poprawek (po akceptacji w przeglądzie).
- **Utrwalenie** — kod zapisany w polu `test_code` przypadku testowego.

Powiązane

- ADR-006, ADR-007.
- test-case-generation-native.md — źródło scenariuszy.

Analiza GUI — pipeline natywny CV/OCR (Epic 3)

Od screenshotu interfejsu do listy komponentów (`GUIComponent`), drzewa layoutu i regresji wizualnej. Detekcja czysto heurystyczna (geometria bbox + słowa kluczowe OCR) — bez modeli ML ani template matchingu. Wszystkie biblioteki obrazowe (OpenCV, Pillow, pytesseract, EasyOCR, scikit-image, scipy, numpy) są **opcjonalne** — orchestrator degraduje się łagodnie (wynik częściowy + ostrzeżenia, bez błędu). Endpointy: uml_gui_examples.md; modele: uml-gui-data-models.md.

Etapy pipeline'u

```
Screenshot GUI (upload, ≤ 20 MB)
  1. Dekodowanie obrazu      (cv2 → Pillow fallback)
  2. OCR                     (Tesseract → EasyOCR fallback, j. pl+en)
  3. Detekcja komponentów    (kontury cv2 + klasyfikacja heurystyczna)
  4. Analiza layoutu         (zagnieżdżanie bbox + przypisanie etykiet)
  → GUIAnalysisOutput (komponenty + bloki OCR + drzewo + ostrzeżenia)
```

Preprocessing (`component_detector._cv2_regions`)

Skala szarości (`cvtColor`) → rozmycie Gaussa (`GaussianBlur` (3,3)) → krawędzie (`Canny` 50/150) → dylatacja (jądro 3×3, 2 iter.) → kontury (`findContours`, `RETR_EXTERNAL`). Dekodowanie: cv2, potem Pillow; brak obu → ostrzeżenie + pusty wynik.

> Uwaga: **brak adaptacyjnego progowania** (thresholding) — pipeline opisany wiernie względem kodu.

OCR (`ocr_service.py`)

- **Tesseract** (główny): `image_to_data(DICT)`, bloki o `confidence ≥ 0.3`.
- **EasyOCR** (fallback): `Reader(["pl","en"], gpu=False)`, gdy Tesseract brak/pusty.
- Brak obu → pusta lista bloków + ostrzeżenie. Blok = `text` + `bbox` + `confidence`.

Detekcja komponentów (`component_detector.classify_region`)

Kontury → filtr geometrii (szer. ≥ 12, wys. ≥ 8, pow. ≥ 200 px^2, `confidence ≥ 0.5`) → klasyfikacja heurystyczna łączająca aspect ratio/powierzchnię ze słowami kluczowymi OCR.

Typ (`GUIComponentType`)	Aktywnie wykrywany	Pewność
`button`	tak	0.8/0.65
`input`	tak	0.65
`dropdown`	tak	0.55
`checkbox`	tak	0.6
`menu`	tak	0.7/0.55
`form`	tak	0.5
`radio` , `link` , `table` , `modal` , `tab`	enum istnieje, brak reguły → zwykle `unknown`	—
`unknown`	fallback	0.3

> Odstępstwo od pierwotnego AC (>80% dla 10 typów): aktywnie klasyfikowane 6 z 12 wartości enuma; pozostałe regiony → `unknown` . Cypress/SikuliX generują kod dla każdego poprawnie sklasyfikowanego typu.

Analiza layoutu (`layout_analyzer.py`)

Zagnieżdżanie przestrzenne wg zawierania bbox: sortowanie po powierzchni (desc) → najmniejszy zawierający rodzic; nieoznaczone komponenty dostają etykiety OCR z okolicy (≤ `_LABEL_GAP = 24 px`); korzeń = syntetyczny `screen` . Relacje: zawieranie button↔form, powiązanie label↔input.

Regresja wizualna (3.10, `visual_regression.py`)

`POST /projects/{id}/gui/compare` porównuje dwa screenshoty:

- **Różnica pikselowa** (`compute_pixel_diff`): SSIM (scikit-image) → `(1-ssim)*100%` ; fallback udział pikseli > 30; fallback 0.0 bez numpy.
- **Delta komponentowa** (`compute_component_delta`): `removed` / `added` / `moved` (≤40 px)/ `layout` (>40 px)/ `text` (podobieństwo < 0.85)/ `color` (zarezerwowane).
- **Status** (`ComparisonStatus`): `major_diff` (delta BREAKING lub pixel ≥ 30%) / `match` (pixel < 2% i brak delt) / `minor_diff` / `error` .
- **Dopasowanie** (`match_components`): `0.4*IoU + 0.3*etykiety + 0.3*typ` ; algorytm węgierski (scipy) lub zachłanny; próg 0.6.

Generowanie PNG SikuliX (domknięcie 3-13)

SikuliX jest image-based — skrypt referuje `Pattern("images/...")` . Dedykowany pipeline `sikulix_pipeline.generate_sikulix_code` (oddzielony od ścieżki Cypress, bo wymaga obrazów):

1. **Linkage step→GUIComponent** (`resolve_step_component`): explicit `step.component` z bbox → `component_id` (lookup) → label-match

(`target` / `target_element` ↔ `GUIComponent.label`, diakrytyki/whitespace → `_`). Brak → krok bez patternu.
2. **Crop**: komponenty pogrupowane po `storage_key` → pobranie screenshotu raz z MinIO → `PatternCropper.crop_all_components` (bbox→PNG, Pillow).
3. **Persystencja**: PNG wgrywane do MinIO (`{tenant}/{project}/sikuli/{test_case_id}/images/{name}`) i odnotowane w `TestCode.pattern_images` JSONB (`[{name, storage_key, component_id}]`). Nazwa PNG (`{ctype}_{label}.png`) spójna między cropperem a generatorem → skrypt referuje realnie wyprodukowane pliki.
4. **Graceful degrade**: bez Pillow → `crop_all_components` zwraca `[]` + warning → skrypt ze statycznymi referencjami, `pattern_images = null` (nie crash).

Patrz ADR-012.

Moduł → serwis → story

Komponent	Serwis	Story
upload screenshotu	`GUIUploadService`	3.8
analiza CV/OCR	`gui/orchestrator.py`	3.8
flows z sekwencji ekranów	`gui/flow_service.py`	3.9
regresja wizualna	`visual_regression.py`	3.10
PNG SikuliX	`sikulix_pipeline.py`, `pattern_cropper.py`	3.13, 3-13

Ograniczenia

- Detekcja heurystyczna wrażliwa na motyw (jasny/ciemny), font, DPI, nakładające się elementy — brak ML/template matchingu.
- Brak twardej walidacji min. rozdzielczości (rozmiar zapisywany, nie odrzucany); formaty szersze niż pierwotne AC (dodatkowo BMP/TIFF).
- Jakość OCR zależy od kontrastu i obecności Tesseract/EasyOCR w środowisku.

Powiązane

- uml-parsing.md — osobny pipeline diagramów UML.
- test-generation-uml-gui.md — flows → Gherkin → kod.
- ADR-012, ADR-008.

Generowanie testów z UML/GUI — pipeline natywny (Epic 3)

Od zrekonstruowanych przepływów użytkownika (`UserFlow`) do scenariuszy BDD (`GherkinFeature`) i kodu testowego (`TestCode`: Cypress / SikuliX). Mapowanie oparte na regułach + szablonach Jinja2. Endpointy: uml_gui_examples.md; modele: uml-gui-data-models.md.

Etapy pipeline'u

```
ParsedUMLModel / GUIScreenshot
  1. Rekonstrukcja user flows (cross-diagram 3.7 / sekwencja ekranów 3.9)
       → UserFlow → FlowStep (z metadanymi relacji/decyzji)
  2. Mapowanie flows → Gherkin (3.11)
       include → Background | extend → Scenario Outline+Examples |
decyzje → Scenarios
       → GherkinFeature (status: generated → validated → approved)
  3. Codegen (3.12 Cypress / 3.13 SikuliX) z zatwierdzonych feature'ów
       → TestCode (Cypress: DOM; SikuliX: image-based + realne PNG)
```

Rekonstrukcja przepływów (3.7 / 3.9)

- **Cross-diagram (3.7, `CrossDiagramAnalyzer`)** — łączy diagramy UML jednego projektu: participant→klasa, klasa→state machine, rekonstrukcja activity/state. Po domknięciu 3-1b czyta również elementy z OCR (`element_kind` / `node_type`).
- **GUI (3.9, `gui/flow_service.py`)** — sekwencja screenshotów → przepływ przez komponenty; `meta` niesie `action_type` / `confidence` (include/extend/decyzje N/A).

`FlowStep` niesie akcję, aktora, cel, oczekiwany rezultat, źródło (`source_diagram_id` / `source_screenshot_id`) oraz **`metadata`** (JSONB, domknięcie 3-11).

Mapowanie Gherkin (3.11, `gherkin_mapper.py`) — domknięcie 3-11

`FlowStep.metadata` przenosi semantykę relacji UML, którą mapper tłumaczy realnie (nie heurystyką):

`metadata.relation_type`	**Wynik w `.feature`**
`include`	sekcja `Background` z relacji include (nie wspólny prefiks Given)
`extend` + `parameters`	`Scenario Outline` + `Examples` (auto-detekcja)

`metadata.relation_type`	**Wynik w `.feature`**
— (z `branch_label`)	rozgałęzienia decyzyjne → osobne `Scenario` (grupowanie po etykiecie)

Szablon `gherkin_feature.jinja2` + `validate_gherkin` wspierają Background/Outline/Examples. Wsteczna zgodność: flows bez `metadata` (legacy) → identycznie jak dotychczas (`flow_to_scenario` + heuristic background). Dodatkowo `_reconstruct_use_case` (use-case produkuje flow — wcześniej nie produkował wcale).

Codegen (3.12 / 3.13)

Z `GherkinFeature` o `status=approved` generowany jest `TestCode`:

- **Cypress (3.12)** — DOM-based (selektory); generyczna ścieżka `generate_code_for_test_cases(framework="cypress")`.
- **SikuliX (3.13)** — image-based; dedykowany `sikulix_pipeline.generate_sikulix_code` wycina realne PNG ze screenshotów (domknięcie 3-13, zob. gui-analysis.md).

`TestCode.framework` = `cypress` | `sikulix`; `pattern_images` (JSONB) tylko dla SikuliX.

Moduł → serwis → story

Komponent	Serwis	Story
rekonstrukcja flows (UML)	`CrossDiagramAnalyzer`	3.7
rekonstrukcja flows (GUI)	`gui/flow_service.py`	3.9
Gherkin mapper	`gherkin_mapper.py`	3.11, 3-11
codegen Cypress	`test_generation/*` (Jinja2)	3.12
codegen SikuliX + PNG	`sikulix_pipeline.py`, `pattern_cropper.py`	3.13, 3-13

Powiązane

- uml-parsing.md, gui-analysis.md — źródła flows.
- traceability-uml-gui.md — macierz źródło→test→kod.
- ADR-006, ADR-012.

Macierz śledzenia UML/GUI — pipeline natywny (Epic 3)

Od elementów źródłowych (UML/GUI) do macierzy łączącej je z przepływami, scenariuszami Gherkin i pokryciem. Budowa regułowa; eksport CSV/XLSX/PDF. Endpointy: uml_gui_examples.md; modele: uml-gui-data-models.md.

> Uwaga: `/projects/{id}/traceability` (Epic 3) to macierz **źródeł UML/GUI** — odrębna od traceability dokumentu (Epic 2, zob. document-analysis.md).

Etapy pipeline'u

```
Elementy źródłowe (UML parsed / GUI components) + UserFlow +
GherkinFeature
  1. _collect_uml_elements / GUI components  → pozycje macierzy
  2. _element_type (czyta element_kind)       → kanoniczny typ (spójny
OCR vs parser)
  3. linkage: element → flow → gherkin       → coverage_status
  4. persystencja: TraceabilityMatrix (source/confidence) + summary
  → GET /traceability (matrix + summary) | /traceability/export
(csv|xlsx|pdf|json)
```

Spójność typów post-3-1b (domknięcie 3-15)

`_element_type` (`traceability_service.py`) czyta `element_kind` **pierwsze** (kanoniczny dyskryminator strukturalny), potem `element_type`, potem `type`. Dzięki temu klasa z OCR (`element_type="class"`) i z XMI (`element_kind="class"`) → obie `"class"` (koniec asymetrii „class vs element"). Mapa `_UML_TYPE_MAP` uzupełniona o tokeny wizualne: `decision` →activity, `enum` →class; `node` dwuznaczny → rozwiązywany parametrem `category` (`activity` / `state_machine`, z `diagram_type`).

Provenance (domknięcie 3-15)

Kolumny `source` (`parser`|`ocr`, VARCHAR(20)) i `confidence` (FLOAT) na `TraceabilityMatrix` (migracja `025`) — pochodzą z `ParsedUMLModel` (diagram-level, aplikowane na wszystkie elementy diagramu). Komponenty GUI → `source=null` (brak podziału parser/OCR). Eksponowane w schemacie `TraceabilityEntryOut`.

Semantyka pokrycia — Gherkin-proxy (AC4, domknięcie 3-15)

Pokrycie = element ↔ przepływ ↔ **zatwierdzony** `GherkinFeature` (`status=approved`):

- `coverage_status = covered` ↔ istnieje `linked_gherkin_id` ze zatwierdzonym feature.
- `code_reference` = nazwa zatwierdzonego feature; `code_framework = "gherkin/bdd"`.
- `test_case_id` → `null` (udokumentowane: `test_cases` wiąże się z `analysis_results`, nie z UML/GUI).

> **Szczere zastrzeżenie:** APPROVED Gherkin to zaakceptowana specyfikacja BDD, **nie** wygenerowany kod testowy (`TestCode`). Realne linkage element→TestCase→TestCode pozostaje follow-upem (wymagałoby nowych relacji). Udokumentowane w docstringach serwisu/modelu/schemy. Patrz ADR-013.

Eksport (AC5, domknięcie 3-15)

`GET /projects/{id}/traceability/export?format=csv|xlsx|json|pdf`:

- **CSV** (separator `;`, UTF-8 BOM) i **JSON** — natywne.
- **XLSX** (`openpyxl`) i **PDF** (`reportlab`) — **funkcjonalne** (przeniesione do main deps, migracja z dev extra — uniknięcie driftu). Lazy-import + HTTP 501 fallback zachowane jako defense-in-depth (testowane przez mock→None).

Moduł → serwis → story

Komponent	Serwis	Story
budowa macierzy	`traceability_service.py`	3.15, 3-15
eksport XLSX/PDF	`traceability_service._build_xlsx/_build_pdf`	3.15, 3-15
persystencja	`TraceabilityMatrix` (+ `source` / `confidence`)	3-15

Powiązane

- uml-gui-data-models.md — `TraceabilityMatrix` + ERD.
- ADR-013, ADR-010.

Modele danych — moduł UML/GUI (Epic 3)

Tabele DB (SQLAlchemy 2.0 async, Postgres) i schematy Pydantic dla modułu UML/GUI (Stories 3.1–3.15). Wszystkie tabele dziedziczą po mixinach `UUIDMixin` (`id: UUID` PK), `TimestampMixin` (`created_at` / `updated_at`); tabele tenantowe dodatkowo `TenantMixin` (`tenant_id` FK→tenants, CASCADE) i są chronione **Row-Level Security** (polityki `tenant_isolation` + `admin_bypass`). Wartości enum z introspekcji żywego kodu.

Diagram ERD (PlantUML)

```
@startuml
hide circle
skinparam linetype ortho

entity uml_diagrams {
  * id : UUID
  --
  format : UMLFormat
  diagram_type : DiagramType
  status : UMLDiagramStatus
}
entity parsed_uml_models {
  * id : UUID
  --
  diagram_id : UUID
  elements/relationships/flows : JSONB
  source : parser|ocr
  confidence : FLOAT
}
entity gui_screenshots {
  * id : UUID
  --
  status : GUIAnalysisStatus
}
entity gui_analysis_results {
  * id : UUID
  --
  screenshot_id : UUID
}
entity gui_components {
  * id : UUID
  --
  component_type : GUIComponentType
  bbox : JSONB
}
entity user_flows {
  * id : UUID
  --
  source_type : FlowSourceType
  status : FlowStatus
}
entity flow_steps {
  * id : UUID
  --
  flow_id : UUID
  metadata : JSONB
}
entity gherkin_features {
  * id : UUID
  --
  source_flow_id : UUID
  status : GherkinStatus
```

```
}
entity test_codes {
  * id : UUID
  --
  framework : cypress|sikulix
  pattern_images : JSONB
}
entity traceability_matrix {
  * id : UUID
  --
  source : parser|ocr
  confidence : FLOAT
  coverage_status : CoverageStatus
}

uml_diagrams ||--|| parsed_uml_models
gui_screenshots ||--o{ gui_analysis_results
gui_analysis_results ||--o{ gui_components
user_flows ||--o{ flow_steps
user_flows ||--o{ gherkin_features
uml_diagrams ..o{ traceability_matrix
gui_screenshots ..o{ traceability_matrix
user_flows ..o{ traceability_matrix
gherkin_features ..o{ traceability_matrix
@enduml
```

Enumy (wartości live)

Enum	Wartości
`UMLFormat`	`xmi`, `plantuml`, `svg`, `pdf`, `raster`
`DiagramType`	`sequence`, `use_case`, `class`, `activity`, `state_machine`
`UMLDiagramStatus`	`pending`, `parsing`, `completed`, `failed`
`GUIComponentType`	`button`, `input`, `dropdown`, `checkbox`, `radio`, `menu`, `link`, `table`, `form`, `modal`, `tab`, `unknown`
`GUIAnalysisStatus`	`pending`, `processing`, `completed`, `failed`
`FlowSourceType`	`uml`, `gui`, `uml_and_gui`
`FlowStatus`	`draft`, `validated`, `tests_generated`
`GherkinStatus`	`generated`, `validated`, `approved`
`TestType`	`unit`, `integration`, `e2e`, `security`, `performance`, `positive`, `negative`, `boundary`, `accessibility`, `semantic`
`TestPriority`	`critical`, `high`, `medium`, `low`
`ComparisonStatus`	`match`, `minor_diff`, `major_diff`, `error`
`CoverageStatus`	`covered`, `partial`, `uncovered`

Tabele (kluczowe pola)

`uml_diagrams` (3.1)

`project_id` FK, `filename`, `storage_key`, `format` (UMLFormat), `diagram_type` (DiagramType, nullable), `raw_content` TEXT, `status` (UMLDiagramStatus), `file_size`, `mime_type`, `error_message` + tenant mixins.

`parsed_uml_models` (3.2–3.6, 3.1b)

`diagram_id` FK (1:1), `model_type`, `participants` / `elements` / `relationships` / `fragments` / `flows` (JSONB), `source` (`parser` | `ocr`), `confidence` FLOAT.

`gui_screenshots` (3.8)

`project_id` FK, `filename`, `storage_key`, `screen_sequence`, `status` (GUIAnalysisStatus), `file_size`, `mime_type`, `error_message` + tenant mixins.

`gui_analysis_results` (3.8)

`screenshot_id` FK, `ocr_text_blocks` / `layout_tree` / `warnings` (JSONB), `processing_time_ms`.

`gui_components` (3.8)

`analysis_id` FK, `sequence`, `component_type` (GUIComponentType), `label`, `bbox` JSONB (`{x,y,width,height}`), `confidence` FLOAT, `attributes` JSONB.

`user_flows` (3.7, 3.9)

`project_id` FK, `source_type` (FlowSourceType), `source_ids` / `steps` / `warnings` (JSONB), `name`, `status` (FlowStatus) + tenant mixins.

`flow_steps` (3.7, 3.9, 3-11)

`flow_id` FK, `sequence`, `action`, `actor`, `target_element`, `expected_result`, `source_diagram_id`, `source_screenshot_id`, `source`, **`metadata` JSONB** (`relation_type` / `parameters` / `branch_label` — domknięcie 3-11, migracja `028`).

`gherkin_features` (3.11)

`project_id` FK, `source_flow_id` FK, `feature_name`, `feature_description`, `scenarios` JSONB, `raw_feature` TEXT, `validation_warnings` JSONB, `status` (GherkinStatus) + tenant mixins.

`test_codes` (3.12, 3.13, 3-13)

`test_case_id` FK, `framework` (`cypress` / `sikulix`), `language`, `code` TEXT, `file_path`, **`pattern_images` JSONB** (`[{name, storage_key, component_id}]` — domknięcie 3-13, migracja `026`; tylko SikuliX) + tenant mixins.

`visual_regression_results` (3.10)

`project_id` FK, `baseline_screenshot_id` / `comparison_screenshot_id` FK, `pixel_diff_percentage` FLOAT, `semantic_diff` / `changes` / `warnings` JSONB, `diff_image_key`, `status` (ComparisonStatus) + tenant mixins.

`traceability_matrix` (3.15, 3-15)

`project_id` FK, `source_type` (`uml` / `gui`), `source_element_id`, `source_element_type`, `source_element_name`, **`source` (`parser` | `ocr`), `confidence` FLOAT** (domknięcie 3-15, migracja `025`), `source_diagram_id` / `source_screenshot_id` / `linked_flow_id` / `linked_gherkin_id` / `test_case_id` FK (SET NULL), `code_reference`, `code_framework`, `coverage_status` (CoverageStatus) + tenant mixins. Indeksy: `(project_id, source_type)`, `(project_id, coverage_status)`, `(source_element_id)`.

Schematy Pydantic (przykłady)

`TraceabilityMatrixOut` (GET /traceability)

```
{
  "elements": [{
    "id": "8b1f…", "source_type": "uml", "source": "parser",
         "confidence": 1.0,
    "source_element_type": "use_case", "source_element_name":
         "Logowanie",
    "linked_flow_id": "a2c4…", "linked_gherkin_id": "f9e1…",
         "test_case_id": null,
    "code_reference": "Feature: Logowanie", "code_framework":
         "gherkin/bdd",
    "coverage_status": "covered"
  }],
  "summary": {
    "uml_total": 12, "gui_total": 8, "uml_coverage_pct": 75.0,
         "gui_coverage_pct": 50.0,
    "code_coverage_pct": 40.0
  },
  "total": 20, "page": 1, "page_size": 20
}
```

`GherkinFeatureOut` (GET /gherkin/{id})

```
{
  "id": "f9e1…", "feature_name": "Logowanie", "source_flow_id": "a2c4…",
  "scenarios": [{"name": "Poprawne logowanie", "tags": [],
    "steps": [{"keyword": "When", "text": "podaję dane"}],
    "examples_headers": [], "examples_rows": []}],
  "status": "approved"
}
```

Golden data (3-14)

Regresja parserów i detekcji jest pokryta golden datasetem w `backend/tests/golden_data/`: **30 UML** (25 PlantUML + 5 XMI, 6/typ), **5 SVG** (visual, `_parse_visual`, `source=ocr`), **15 GUI PNG** + **10 par** visual_regression. Stage CI `golden-regression` + artefakt `golden-accuracy.json` (threshold-gate); testy obrazowe image-gated (skip bez cv2/Tesseract). Patrz ADR-008.

Powiązane

- uml-parsing.md, gui-analysis.md, test-generation-uml-gui.md, traceability-uml-gui.md.

Parsowanie UML — pipeline natywny (Epic 3)

Od diagramu UML (PlantUML/XMI lub wizualnego SVG/PDF/raster) do ustrukturyzowanego `ParsedUMLModel`, którym zasilane są rekonstrukcja przepływów (3.7), porównanie z kodemu (3.4) i macierz śledzenia (3.15). Parsowanie oparte na regułach (gramatyka PlantUML, XML `defusedxml`, OCR tekstu + geometria kształtów). Endpointy: uml_gui_examples.md; modele: uml-gui-data-models.md.

Etapy pipeline'u

```
Plik UML (upload)
  1. detect_format()   → UMLFormat (plantuml | xmi | svg | pdf | raster)
  2. detect_diagram_type() → DiagramType (sequence | use_case | class |
activity | state_machine)
  3. parse()
       Tier 1 (plantuml/xmi)  → pełna ekstrakcja strukturalna
(uczestnicy, elementy, relacje)
       Tier 2 (svg/pdf/raster) → analiza wizualna (OCR + kształty),
source=ocr, confidence
  4. ParsedUMLModel → persystencja JSONB → downstream (3.7 cross-diagram,
3.4 compare-code, 3.15)
```

Każdy model utrwalany w `parsed_uml_models` (JSONB); `source` (`parser`|`ocr`) i `confidence` sygnalizują wierność ekstrakcji.

Formaty (macierz: format × typ diagramu)

Format	Typ diagramu	Status
PlantUML (`@startuml`)	wszystkie 5	**pełne** — strukturalna ekstrakcja (Tier 1)
XMI (UML 2.x, OMG/Eclipse)	wszystkie 5	**częściowe/pełne** — klasy pełne, pozostałe częściowe (zależność od wersji/narzędzia)
SVG	wszystkie 5	**OCR** — `defusedxml` wektor, najwyższa wierność Tier 2, bez deps binarnych
PDF	wszystkie 5	**OCR** — PyMuPDF + OCR; graceful degrade bez deps
PNG/JPEG (raster)	wszystkie 5	**OCR** — cv2 + Tesseract/EasyOCR; graceful degrade bez deps

> Tier 2 (SVG/PDF/raster, Story 3.1b + domknięcie 3-1b): analiza opiera się na OCR tekstu i wnioskowaniu kształtów (prostokąt→klasa, owal→use case, romb→decyzja, strzałka→relacja). Wynik oznaczony `source: ocr` z

> `confidence`. Akceptację rozszerzeń definiuje `UML_ALLOWED_EXTENSIONS` (`upload_service.py`).

Wykrywanie formatu i typu

- **Format** (`detect_format` / `detect_format_from_content`, `uml/upload_service.py`): rozszerzenie (`.puml` / `.xmi` / `.svg` / `.pdf` / `.png`) → magic bytes (`%PDF`, `\x89PNG`, `<svg`, `@startuml`, `<?xml` / `<xmi`) → heurystyka (`<` → XMI, inaczej PlantUML).
- **Typ** (`detect_diagram_type`): PlantUML — markery strukturalne + słowa kluczowe (`class` / `interface` / `enum`, `start` / `stop` / `if`, `[*]` / `state`); XMI — nazwy elementów regex; wizualne — skład kształtów (`classify_visual_diagram_type`, `uml/visual/shapes.py`).

Adapter visual→structural (domknięcie 3-1b)

Kluczowa decyzja: parser wizualny (`build_visual_model`, `uml/visual/shapes.py`) emituje **zarówno** pola wizualne (`element_type`, `bbox`), **jak i strukturalne** — tak, żeby downstream pisany pod schema strukturalne widział elementy z OCR:

- klasy (rect+CLASS) → `element_kind` (`class` / `interface` / `enum`) + stabilne `id` (`v{idx}`)
- węzły activity → `node_type` (`start` / `stop` / `action` / `decision`)
- węzły state → `is_initial` / `is_final`
- relacje → rozwiązywane `source_id` / `target_id` (obok name-based `source` / `target`)

Bez tego cross-diagram (3.7) i compare-code (3.4) po cichu dostawały puste wyniki. Heurystyka start/stop z topologii DAG + etykiet (OCR nie widzi wypełnionego kółka). Patrz ADR-011.

compare-code (3.4 + domknięcie 3-1b)

`POST /projects/{id}/uml/{diagram_id}/compare-code` porównuje klasy z diagramu z kodemu projektu (`logic_structures`, Epic 1). Dyskryminacja `interface` / `enum` po `element_kind`.
`ClassMatch.attribute_match_pct` / `method_match_pct: float | None`: przy zerowej liczbie成员 ekstrahowanych z diagramu (inherentne dla OCR) comparator zwraca `null`, a nie vacuous `100%` — uczciwe sygnalizowanie niepełnej ekstrakcji.

Ograniczenia (udokumentowane, szczere)

- **OCR nie ekstrahuje成员 klasy** (atrybuty/metody) — inherentne; compare-code raportuje `*_match_pct = null`.
- **Dyskryminacja interface/enum z OCR niewyprowadzalna** — wszystkie klasy z OCR to `element_kind="class"` (brak markera wizualnego).

- `classify_visual_diagram_type` **nigdy nie zwraca** `state_machine` (diamond → activity×3 > state×1) — wizualne diagramy stanów bywają misklasyfikowane jako activity; golden dataset odzwierciedla empirycznie detekowany typ.
- XMI: zależność od wersji (2.1/2.5) i narzędzia; bezpieczny XML (`defusedxml`).

Moduł → serwis → story

Komponent	Serwis	Story
upload + detect	`UMLUploadService`	3.1
parser sekwencji/use-case/klas/aktywności/stanów	`uml/parsers/*`	3.2–3.6
parser wizualny (Tier 2)	`uml/visual/*` (`build_visual_model`)	3.1b, 3-1b
cross-diagram → flows	`CrossDiagramAnalyzer`	3.7
compare-code	`class_code_comparator`	3.4, 3-1b

Powiązane

- gui-analysis.md — osobny pipeline screenshotów GUI.
- test-generation-uml-gui.md — flows → Gherkin → kod.
- ADR-011, ADR-008.

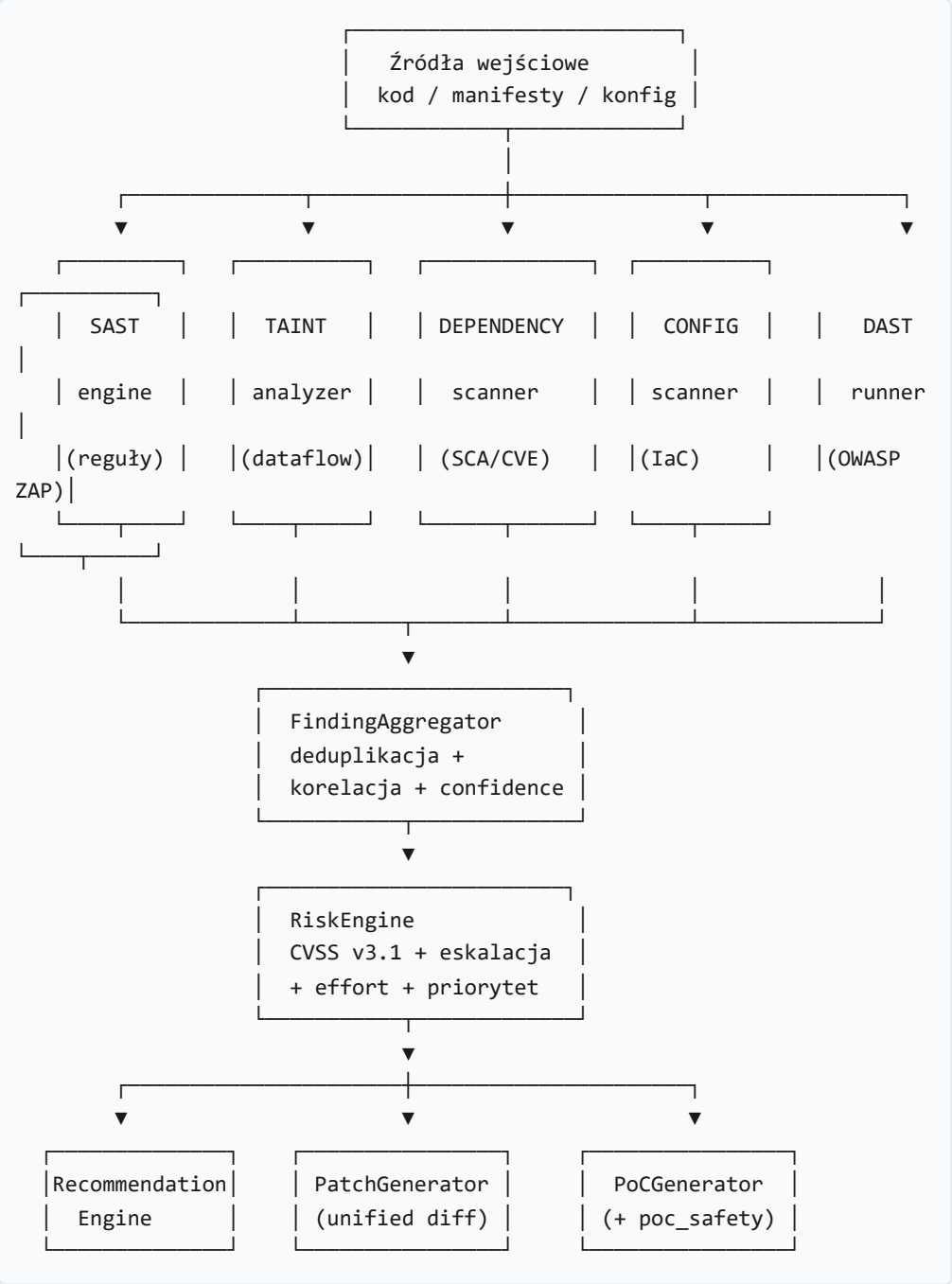
Architektura modułów bezpieczeństwa — przegląd

> Moduły bezpieczeństwa TestGEN (Epic 4). Wersja odpowiada implementacji w `backend/app/services/security_scan/`.

1. Cel modułu

Moduł bezpieczeństwa TestGEN realizuje kompleksowy, wielowarstwowy pipeline analizy bezpieczeństwa kodu źródłowego, zależności i konfiguracji infrastruktury. Silniki są deterministyczne (oparte na regułach i analizie przepływu danych): działają offline, bez zależności od usług zewnętrznych, z gwarancją powtarzalności wyników.

2. Diagram pipeline'u



3. Moduły i ich odpowiedzialności

Moduł	Plik	Rola
SAST engine	`sast_engine.py`	Statyczna analiza wzorców (reguły regex per język)
Taint analyzer	`taint_analyzer.py`	Analiza przepływu danych source→sink
Dependency scanner	`dependency_scanner.py`	SCA — parsowanie manifestów, dopasowanie CVE, SBOM

Moduł	Plik	Rola
Config scanner	`config_scanner.py` + `config_scanners/`	Docker / Kubernetes / Terraform
DAST runner	`dast_runner.py`	Dynamiczna analiza przez OWASP ZAP
Finding aggregator	`finding_aggregator.py`	Deduplikacja, korelacja, scoring zaufania
Risk engine	`risk_engine.py`	CVSS, eskalacja, szacowanie wysiłku, priorytetyzacja
CVSS calculator	`cvss_calculator.py`	Czysty kalkulator CVSS v3.1
CVE pipeline	`cve_pipeline.py`	Aktualizacja bazy CVE z NVD/OSV/GitHub
Recommendation engine	`recommendation_engine.py`	Rekomendacje naprawcze
Patch generator	`patch_generator.py`	Generowanie patchy git-apply
PoC generator	`poc_generator.py` + `poc_safety.py`	Bezpieczne PoC

4. Wspólne typy wyników

Wszystkie silniki produkują rekordy `SecurityFinding` (model `app/models/security_finding.py`) o ujednoliconej strukturze: typ podatności, severity, CWE, kategoria OWASP, lokalizacja (plik/linia), confidence (0.0–1.0), źródło oraz metadane (np. ścieżka taint, payload PoC).

5. Przepustowość i ograniczenia

- SAST/Taint: limit 2 MB na plik (`_MAX_CONTENT_BYTES = 2_000_000`).
- Config: limit 1 MB na plik.
- DAST: domyślny limit skanu 1800 s (30 min).

6. Powiązane dokumenty

- 02 — SAST engine
- 03 — Taint analyzer
- 04 — Dependency scanner
- 05 — Config scanner
- 06 — DAST runner
- 07 — Agregator i risk engine
- 09 — Modele danych

SAST engine — silnik statycznej analizy

Plik: `app/services/security_scan/sast_engine.py` Klasa publiczna: `SASTEngine`

1. Cel

SAST engine wykonuje statyczną analizę kodu opartą na wzorcach (reguły regex per język). Wykrywa typowe klasy podatności (SQLi, XSS, CSRF, IDOR/authz, hardcoded secrets, command/code injection, path traversal, weak crypto) przeszukując kod linia po linii i dopasowując reguły z rejestru reguł.

2. Interfejs

```
SASTEngine(db=None)

def scan_code(code: str, language: str | None, file_path: str |
        None=None) -> list[RawFinding]
async def run_sast(project_id, tenant_id, scan_id) ->
        list[SecurityFinding]
```

- `scan_code` — czysta, testowalna funkcja: wejście to surowy kod, wyjście to lista `RawFinding`. Nie wymaga DB ani sieci.
- `run_sast` — pełny przebieg: ładuje pliki projektu, uruchamia analizę regułową i zapisuje `SecurityFinding`.

3. Algorytm (ścieżka native)

1. Załaduj pliki źródłowe projektu (limit 2 MB/plik, filtrowanie po języku).
2. Dla każdej linii każdego pliku:
   - Pomiń komentarze/puste linie, chyba że reguła jawnie celuje w komentarze (gdy regex zawiera `#` lub `//`).
   - Przetestuj linię względem wszystkich reguł właściwych dla języka (`rules_for_language`).
   - Po dopasowaniu wzorca oblicz confidence: bazowa pewność reguły + `confidence_weight` wzorca + korekty kontekstowe.
   - Utwórz `RawFinding` z CWE, kategorią OWASP i (ewentualnym) override severity.
3. Zwróć listę znalezisk.

4. Korekty pewności (confidence)

- **+0.15** — obecność markerów danych użytkownika (`request`, `input`, `argv`, `params`, `req.`, `$_`).
- **−0.05** — linie z dużą liczbą literałów (mniejsze ryzyko).
- Wynik końcowy przycinany do zakresu **[0.1, 0.99]**.

5. Mapa override severity

vulnerability_type	severity
command_injection	critical
sql_injection	critical
xss	high
weak_cryptography	medium

6. Wejścia i wyjścia

- **Wejście:** kod źródłowy (str), język, ścieżka pliku.
- **Wyjście:** `list[RawFinding]` → mapowane na `SecurityFinding` przy zapisie (`_to_finding`).

7. Zależności

- `rules/` (rejestr reguł, `RawFinding`, `SEVERITY_ORDER`)
- `models/security_finding.py` (`SecurityFinding`, `ScanType`, `Severity`, `AnalysisSource`)

8. Wydajność

Analiza liniowa O(linie × reguły) z wczesnym pomijaniem komentarzy. Brak parsowania AST w ścieżce krytycznej — reguły operują na regex, co zapewnia przewidywalny, szybki przebieg offline. Szczegóły reguł: katalog reguł.

Taint analyzer — analiza przepływu danych

Plik: `app/services/security_scan/taint_analyzer.py` Klasa publiczna: `TaintAnalyzer`

1. Cel

Taint analyzer śledzi przepływ niezaufanych danych od **źródła** (source, np. `request.GET`) do **niebezpiecznej operacji** (sink, np. `cursor.execute`) i raportuje podatność, jeśli dane docierają do sinka bez adekwatnej sanityzacji. Analiza jest „forward", w kolejności linii, w zakresie pojedynczego pliku.

2. Interfejs

```
def analyze_code(code: str, language: str | None, file_path=None) ->
        list[TaintPath]
async def run_taint_analysis(project_id, tenant_id, scan_id) ->
        list[SecurityFinding]
```

3. Struktury danych

- **TaintPathStep** — jeden krok ścieżki: `file_path`, `line`, `variable`, `operation` (fragment kodu), `node_type` ∈ {source, transform, sanitizer, sink}.
- **TaintPath** — kompletny przepływ source→sink: `source_name`, `sink_name`, `sink_type`, `taint_type`, `vulnerability_type`, `cwe_id`, `severity`, `variable`, `steps[]`, `sanitizer_name`, `sanitizer_adequate`, zakres linii.
- **_TaintState** (wewn.) — stan skażenia zmiennej: `source_name`, `taint_type`, `steps[]`, `protected_against` (frozenset typów sinków, przed którymi chroni zastosowany sanitizer), `sanitizer_name`.

4. Algorytm

1. Pobierz definicje taint dla języka (sources / sinks / sanitizers — patrz taint-definitions).
2. Przejście „forward" po liniach:
   - **Sink first** — jeśli linia zawiera sink i dociera do niego skażona zmienna bez adekwatnego sanitizera → zapisz znalezisko.
   - **Source** — jeśli linia zawiera source, propaguj skażenie do zmiennej po lewej stronie przypisania (nowy `TaintPathStep`).
   - **Propagacja** — jeśli prawa strona jest już skażona, propaguj skażenie; jeśli zastosowano sanitizer, oznacz, dla jakich `sink_type` skażenie jest usunięte.
3. Confidence: **0.55** gdy sanitizer istnieje, ale jest nieadekwatny; **0.7** gdy brak sanitizera.

4. Pełna ścieżka (source→transforms→sink) trafia do `finding_metadata.taint_path` jako JSON.

5. Detekcja przypisań

- `_ASSIGN_RE` dopasowuje pojedyncze `=` (nie `==`, `+=` itd.) z opcjonalnymi modyfikatorami (`let`, `var`, `public` ...).
- `_IDENT_RE` ekstrahuje nazwę zmiennej.
- Dopasowanie źródła: `NamedTaintSource.matches(rhs)`.

6. Nieadekwatna sanityzacja

Kluczowa właściwość: sanitizer chroni tylko przed określonymi typami sinków (`protects_against`). Np. `html.escape` chroni przed sinkiem `html`, ale NIE przed sinkiem `sql`. Jeśli dane są „sanityzowane" niewłaściwym sanitizerem dla danego sinka, ścieżka jest flagowana z `sanitizer_adequate = False`.

7. Zależności

- `taint_definitions/` (`sources_for_language`, `sinks_for_language`, `sanitizers_for_language`)
- `rules/` (`owasp_for`)
- `models/security_finding.py`

Dependency scanner — skaner zależności (SCA)

Plik: `app/services/security_scan/dependency_scanner.py` Klasa publiczna: `DependencyScanner`

1. Cel

Software Composition Analysis (SCA): parsuje manifesty zależności pięciu ekosystemów, dopasowuje pakiety do bazy CVE i generuje SBOM (CycloneDX + SPDX).

2. Obsługiwane ekosystemy i parsery

Ekosystem	Pliki	Parser
pypi	`requirements*.txt`, `pyproject.toml`	`_parse_requirements_txt`, `_parse_pyproject_toml`
npm	`package.json`, `package-lock.json`	`_parse_package_json`, `_parse_package_lock`
maven	`pom.xml`, `build.gradle`	`_parse_pom_xml`, `_parse_build_gradle`
nuget	`*.csproj`, `packages.config`	`_parse_csproj`, `_parse_packages_config`
composer	`composer.json`	`_parse_composer_json`

Dopasowanie pliku idzie od najbardziej specyficznego wzorca (requirements*, pyproject.toml, package-lock.json, package.json, pom.xml, build.gradle, .csproj, packages.config, composer.json).

3. Interfejs

```
def parse_file(filename: str, content: str) -> list[Dependency]
async def scan_dependencies(project_id, tenant_id, scan_id) ->
        tuple[list[SecurityFinding], dict]
def generate_cyclonedx_sbom(dependencies, cve_map) -> dict
def generate_spdx_sbom(dependencies, cve_map) -> dict
```

`scan_dependencies` zwraca krotkę `(findings, sbom_payload)`, gdzie `sbom_payload = {"cyclonedx": {...}, "spdx": {...}}`.

4. Struktury danych

- **Dependency** — `name`, `version | None`, `ecosystem`, `is_direct`, `license | None`.

- **CVEEntry** — `cve_id`, `cvss_score`, `description`, `affected_versions`, `fixed_version`, `source_db` ∈ {osv, nvd, snyk}.
- **CVEProvider** (Protocol) — `async query(name, version, ecosystem)`.
  - `LocalCacheCVEProvider` — czyta z tabeli `cve_database` (offline, domyślny).
  - `NullCVEProvider` — pusta lista (fallback testowy).

5. Algorytm

1. Wykryj ekosystem po nazwie pliku → wywołaj właściwy parser.
2. Dla każdej zależności odpytaj `CVEProvider`.
3. Deduplikuj CVE po ID, zachowaj najwyższy CVSS i znane wersje naprawcze.
4. Filtruj CVE z CVSS **≥ 4.0** (`_FINDING_CVSS_THRESHOLD`).
5. Mapuj CVSS na severity: ≥9.0 critical, ≥7.0 high, ≥4.0 medium, reszta low.
6. Wygeneruj SBOM (CycloneDX + SPDX) z komponentami, wersjami, PURL i referencjami do podatności.
7. Auto-rekomendacja: „Zaktualizuj {pkg} z {old} do {fixed}" lub „Brak poprawki… rozważ zamiennik".

6. Pasma severity (CVSS)

Pasmo	Severity
[9.0, 10.0]	critical
[7.0, 9.0)	high
[4.0, 7.0)	medium
[0, 4.0)	low

7. Tryb offline

Domyślny `LocalCacheCVEProvider` korzysta wyłącznie z lokalnej tabeli `cve_database` (zasilanej przez CVE pipeline), dzięki czemu skan jest powtarzalny i nie wymaga sieci.

8. Zależności

- SQLAlchemy (`CVEDatabase`), `ElementTree` (XML), `tomllib` (TOML), `json`.
- `models/security_finding.py`.

Config scanner — skaner konfiguracji infrastruktury

Plik orkiestratora: `app/services/security_scan/config_scanner.py` Skanery technologiczne: `app/services/security_scan/config_scanners/` Klasa publiczna: `ConfigScanner`

1. Cel

Wykrywa błędy konfiguracji (misconfigurations) w plikach Infrastructure-as-Code: Docker, Kubernetes i Terraform. Orkiestrator klasyfikuje plik po nazwie i kieruje go do właściwego skanera technologicznego.

2. Klasyfikacja plików

Wzorzec nazwy	Technologia
`dockerfile*`, `docker-compose*.yml`	docker
`*.tf`, `*.tf.json`	terraform
`*.yaml`, `*.yml` (nie Docker/TF)	kubernetes

3. Interfejs

```
def scan_content(content: str, filename: str) -> list[ConfigFinding]
async def scan_config(project_id, tenant_id, scan_id) ->
        list[SecurityFinding]
```

`ConfigFinding` (`config_scanners/base.py`): `rule_id`, `config_type`, `severity`, `description`, `recommendation`, `fix_example`, `file_path`, `line`, `vulnerability_type`, `cwe_id`, `source`, `extra`.

Interfejs skanerów: `IConfigScanner` (Protocol) — właściwość `config_type` + `scan(content, filename) -> list[ConfigFinding]`.

4. Reguły per technologia

Docker (`docker_scanner.py`)

`LATEST_TAG` (CWE-1104, medium), `ROOT_USER` (CWE-250, high), `EXPOSED_PORT` (CWE-668, medium — porty wrażliwe 22/23/3306/5432/6379/27017/9200/5984), `HARDCODED_SECRET` (CWE-798, critical), `SUDO_IN_DOCKER` (CWE-250, high), `NO_HEALTHCHECK` (CWE-754, low), `PRIVILEGED_CONTAINER` (compose, CWE-250, critical).

Kubernetes (`kubernetes_scanner.py`)

Analizuje workloady (Pod/Deployment/StatefulSet/DaemonSet/ReplicaSet/Job/CronJob): `DEFAULT_NAMESPACE` (CWE-1188, low), `HOST_NETWORK` (CWE-668, high), `PRIVILEGED_CONTAINER` (CWE-250, critical), `RUN_AS_ROOT` (CWE-250, high), `NO_RESOURCE_LIMITS` (CWE-770, medium).

Terraform (`terraform_scanner.py`)

Regex/HCL: `PUBLIC_S3_BUCKET` (CWE-732, high), `OPEN_SECURITY_GROUP` (CWE-284, critical — `0.0.0.0/0`), `PLAINTEXT_SECRET` (CWE-312, medium), `NO_ENCRYPTION` (CWE-311, high — dla `aws_s3_bucket` / `aws_db_instance` / `aws_ebs_volume`), `NO_LOGGING` (CWE-778, medium).

5. Algorytm orkiestratora

1. Klasyfikuj plik → wybierz skaner.
2. Zbierz `ConfigFinding`.
3. Deduplikuj po `(file_path, line, rule_id)`.
4. Mapuj na `SecurityFinding`, kategoria OWASP = `A05:2021-Security Misconfiguration`. Confidence: 0.85.

6. Limity

- 1 MB na plik (`_MAX_CONTENT_BYTES = 1_000_000`).

Pełna lista reguł konfiguracji: ten plik oraz golden data w `backend/tests/golden_data/security/misconfigurations/`.

DAST runner — dynamiczna analiza (OWASP ZAP)

Plik: `app/services/security_scan/dast_runner.py` Klasa publiczna: `DASTRunner`

1. Cel

Orkiestruje cykl życia OWASP ZAP w celu dynamicznego testowania uruchomionej aplikacji: start → gotowość → autentykacja → spider → active scan → alerty → zatrzymanie. Patrz też pipeline DAST.

2. Interfejs

```
async def run_dast(project_id, tenant_id, scan_id, target_url,
         auth_config) -> list[SecurityFinding]
```

3. Struktury danych

- **AuthConfig** — `auth_type` ∈ {form, token, basic} + `credentials` (login_url, username_field, password_field, token, username, password). Fabryka `from_dict()` waliduje typ i ekstrahuje dane.
- **DASTFinding** — `url`, `method`, `vulnerability_type`, `severity`, `cwe_id`, `evidence`, `plugin_id`, `request`, `response`, `recommendation`, `extra`.
- **ZAPController** (Protocol) — wstrzykiwalny kontroler kontenera + REST API (umożliwia testy z atrapą).

4. Algorytm

1. Walidacja `target_url` (wymagany).
2. Start kontenera ZAP (lub wstrzyknięty mock).
3. Oczekiwanie na gotowość (polling `/JSON/core/view/version/`, limit 120 s).
4. Konfiguracja autentykacji (form/token/basic) przez ZAP Replacer API.
5. Spider (crawl) na `target_url`.
6. Active scan; polling statusu do 100% lub timeout (domyślnie 1800 s).
7. Pobranie alertów (JSON).
8. Sanityzacja request/response (poniżej).
9. Mapowanie alertów ZAP → `DASTFinding` → `SecurityFinding`.

5. Sanityzacja danych wrażliwych

- Nagłówki wrażliwe → `[REDACTED]`: `authorization`, `cookie`, `set-cookie`, `x-api-key`, `x-auth-token`, `proxy-authorization`.
- `_TOKEN_RE` zastępuje wartości po `password` / `token` / `api_key` / `access_token` → `[REDACTED]`.

6. Mapowanie ZAP → CWE

`_PLUGIN_TO_CWE` jako fallback gdy alert nie ma CWE, np.: 40018→CWE-89 (SQLi), 40012→CWE-79 (Reflected XSS), 90020→CWE-78 (OS Command Injection). Ryzyko ZAP (high/medium/low/informational) → `Severity`.

7. Stałe

- `_DEFAULT_SCAN_TIMEOUT = 1800` (30 min).
- `_DEFAULT_READY_TIMEOUT = 120` (2 min).

8. Zależności

OWASP ZAP (`owasp/zap2docker-stable`), Docker SDK, `httpx`.

Agregator wyników i risk engine

Pliki: `finding_aggregator.py`, `risk_engine.py`, `cvss_calculator.py`

CZĘŚĆ A — FindingAggregator

1. Cel

Konsoliduje znaleziska ze wszystkich skanerów: deduplikuje, koreluje i wylicza spójny wskaźnik zaufania (confidence).

2. Interfejs

```
async def aggregate(project_id, scan_ids, db) -> AggregationResult
def aggregate_findings(findings) -> AggregationResult
```

`AggregationResult` — `findings`, `summary`, `correlations`, `dedup_count`.

3. Deduplikacja

Klucz: `(file_path, line_start, vulnerability_type, cwe_id)`. Zachowuje pierwsze wystąpienie, scala duplikaty (wyższy confidence, dziedziczenie `taint_path` / `recommendation`), śledzi wszystkie `ScanType` per klucz.

4. Korelacja

Dopasowanie SAST↔DAST/TAINT po równości CWE lub nakładaniu się `vulnerability_type` (np. „sql_injection" w „SQL Injection at…"). `Correlation` — `sast_finding_id`, `dast_finding_id`, `correlation_type` ∈ {sast_dast, sast_taint}, `confidence_boost`.

5. Scoring zaufania

Bazowa pewność narzędzia:

Narzędzie	Confidence bazowy
DEPENDENCY	0.90
DAST	0.85
TAINT	0.80
CONFIG	0.75
SAST	0.70

- Boost za liczbę źródeł: `min(max(n-1,0),3) * 0.05` (maks. +0.15).
- Boost korelacji: **+0.20** SAST↔DAST, **+0.10** SAST↔TAINT (cap 0.20).

- Wynik: `min(base + source_boost + correlation_boost, 1.0)`.

Sortowanie: severity DESC, confidence DESC. Summary: `{total_findings, severity_counts, source_counts}`.

CZĘŚĆ B — RiskEngine

1. Cel

CVSS scoring, eskalacja severity, szacowanie wysiłku naprawy i priorytetyzacja.

2. Interfejs

```
async def calculate_risk(project_id, db) -> RiskAssessment
def assess(findings) -> RiskAssessment
```

`RiskAssessment` — `prioritized_findings`, `cvss_distribution`, `top_10`, `effort_estimates`.

3. Algorytm

1. Brak CVSS → wyprowadź z CWE (`cvss_calculator.default_vector_for`).
2. Brak OWASP → mapuj z CWE (`_OWASP_BY_CWE`).
3. Oszacuj effort z CWE (`_EFFORT_BY_CWE`): low (sanityzacja/update), medium (refaktor), high (przeprojektowanie auth).
4. Reguły eskalacji (bump +1 poziom severity):
   - SAST + DAST razem i severity ≠ CRITICAL → „Potwierdzona dynamicznie".
   - DEPENDENCY z CVSS ≥ 9.0 i severity ≠ CRITICAL → „Zależność z publicznym exploitem".
5. Wyrównaj severity z pasmem CVSS (bierze wyższe).
6. Sortuj `(-severity, -confidence, effort, -cvss)` → przypisz `priority` (1 = najwyższy).

4. Mapowanie CWE → effort

- **low:** CWE-89, 79, 352, 22, 798, 259, 1104, 611
- **medium:** CWE-78, 94, 502, 668, 918, 200
- **high:** CWE-639, 862

CZĘŚĆ C — CVSSCalculator

Czysty kalkulator CVSS v3.1 (`cvss_calculator.py`). `CVSSVector` koduje 8 metryk bazowych (AV, AC, PR, UI, S, C, I, A). `calculate_cvss_base_score` implementuje ISS, Impact, Exploitability i zaokrąglanie wg specyfikacji FIRST. `default_vector_for(cwe, severity)` zwraca kanoniczne wektory NVD, np.:

CWE	Wektor	Wynik
CWE-89 (SQLi)	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H	9.8
CWE-79 (XSS)	AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N	6.1
CWE-352 (CSRF)	AV:N/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N	6.5

Referencja: FIRST CVSS v3.1 Specification.

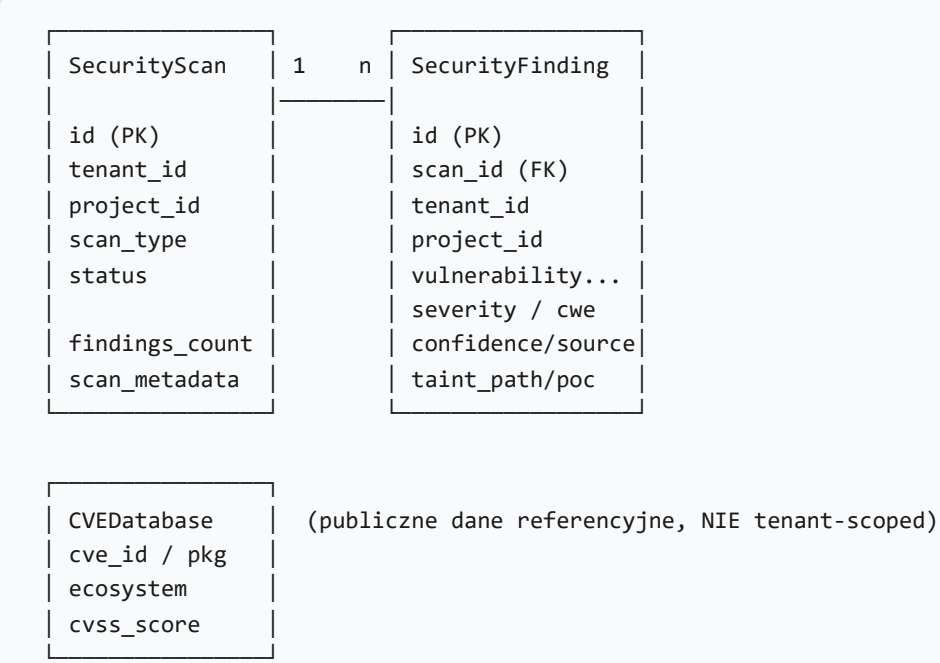
Modele danych — moduł bezpieczeństwa

Plik: `app/models/security_finding.py`

1. Enumy

- **ScanType:** SAST, TAINT, DEPENDENCY, CONFIG, DAST
- **Severity:** CRITICAL, HIGH, MEDIUM, LOW, INFO
- **AnalysisSource:** źródło wykrycia znaleziska
- **FindingStatus:** OPEN, ACKNOWLEDGED, FIXED, FALSE_POSITIVE
- **ScanStatus:** PENDING, RUNNING, COMPLETED, FAILED

2. Diagram ERD (uproszczony)



3. Tabela SecurityScan

Kolumna	Typ	Uwagi
id	UUID	PK
tenant_id	UUID	FK, index
project_id	UUID	FK, index
scan_type	Enum ScanType	index
status	Enum ScanStatus	default PENDING, index
task_id	str | None	id zadania Celery
target_url	str | None	1000 znaków (DAST)
started_at / completed_at	DateTime	

Kolumna	Typ	Uwagi
findings_count	int	
error_message	text	
scan_metadata	JSONB	np. payload SBOM
created_at / updated_at	TimestampMixin	

4. Tabela SecurityFinding

Kluczowe kolumny: `id` (PK), `tenant_id`, `project_id` (index), `scan_id` (FK, index), `scan_type` (index), `vulnerability_type` (index), `severity` (index), `cwe_id`, `owasp_category`, `file_path` (index), `line_start`, `line_end`, `code_snippet` (text, przycinany do 500 zn.), `description`, `confidence` (float 0–1), `source` (Enum AnalysisSource, default NATIVE), `status` (Enum FindingStatus, default OPEN, index), `taint_path` (JSONB), `recommendation`, `finding_metadata` (JSONB).

Pola risk engine (Story 4.8): `cvss_score`, `cvss_vector`, `priority` (1 = najwyższy), `effort` (low/medium/high), `escalation_reason`.

Pola PoC (Story 4.10): `poc_payload`, `poc_steps` (JSONB), `poc_expected_effect`.

5. Tabela CVEDatabase

Publiczne dane referencyjne (NIE tenant-scoped): `id` (PK), `cve_id` (index), `package_name` (index), `ecosystem` (index), `cvss_score`, `severity`, `description`, `affected_versions`, `fixed_version`, `published_date`, `source_db` (default "osv", ∈ {nvd, osv, snyk}).

Katalog reguł — SQL Injection (CWE-89)

Plik reguł: `app/services/security_scan/rules/sqli_rules.py` OWASP: A03:2021-Injection · CWE: CWE-89

Opis klasy podatności

SQL Injection powstaje, gdy zapytanie SQL jest budowane przez konkatenację lub interpolację z niezaufanymi danymi, zamiast przez zapytania parametryzowane. Atakujący może zmodyfikować strukturę zapytania (np. `' OR '1'='1`).

Reguły

SQLI-STRING-CONCAT (wszystkie języki, critical, conf 0.65)

Wzorzec:

```
(?=.*["'])(?=.*\b(SELECT|INSERT|UPDATE|DELETE)\b)(?=.*(\+|\.format\s*\(|["']\s*%|f["']|\$\{|\{[A-Za-z_]))
```

Wykrywa SQL budowane przez konkatenację/format z danymi wejściowymi.

PY-CURSOR-FSTRING (Python, critical, conf 0.7)

```
\.execute\s*\(\s*(f["']|["'][^)]*\+|["'][^)]*%[^,)]*$)
```

`cursor.execute` z f-stringiem/format zamiast parametrów.

JAVA-STATEMENT-CONCAT (Java, critical, conf 0.7)

```
\.execute(Query|Update)?\s*\(\s*["'].*\+|createStatement\s*\(
```

JS-QUERY-CONCAT (JS/TS, critical, conf 0.65)

```
\.query\s*\(\s*([`"'].*\$\{|["'].*\+)
```

CS-SQLCOMMAND-CONCAT (C#, critical, conf 0.7)

```
new\s+SqlCommand\s*\(\s*["'].*\+|ExecuteReader\s*\(\s*["'].*\+
```

Przykład podatny (Python)

```
cursor.execute(f"SELECT * FROM users WHERE id = {user_id}")
```

Przykład naprawiony (Python)

```
cursor.execute("SELECT * FROM users WHERE id = %s", (user_id,))
```

Rekomendacja

Używaj zapytań parametryzowanych / ORM zamiast sklejania łańcuchów. W Java — `PreparedStatement` z `?`; w C# — `SqlParameter`/parametry `@`.

Katalog reguł — Cross-Site Scripting (CWE-79)

Plik reguł: `app/services/security_scan/rules/xss_rules.py` OWASP: A03:2021-Injection · CWE: CWE-79

Opis klasy podatności

XSS powstaje, gdy niesanityzowane dane użytkownika trafiają do wyjścia HTML. Wyróżniamy: reflected (odbity), stored (trwały) i DOM-based.

Reguły

JS-DOM-SINK (JS/TS, high, conf 0.55)

```
\.innerHTML\s*=|\.outerHTML\s*=|document\.write\s*\(|\$\
([^)]*\)\.html\s*\(|insertAdjacentHTML\s*\(
```

Niebezpieczny sink DOM (innerHTML/document.write/$.html).

JS-FRAMEWORK-RAW-HTML (JS/TS, high, conf 0.6)

```
dangerouslySetInnerHTML|v-html\s*=|bypassSecurityTrust(Html|Script)
```

Surowy HTML w React/Vue/Angular.

PY-TEMPLATE-RAW (Python, high, conf 0.55)

```
render_template_string\s*\(|\bMarkup\s*\
(|\|\s*safe\b|autoescape\s*=\s*False
```

JAVA-RESPONSE-WRITE (Java, high, conf 0.5)

```
getWriter\s*\(\s*\)\s*\.\s*(write|print)|out\.print(ln)?\s*\(
```

CS-RESPONSE-RAW (C#, high, conf 0.55)

```
Response\.Write\s*\(|Html\.Raw\s*\(
```

Przykład podatny (JavaScript, DOM-based)

```
element.innerHTML = location.hash.substring(1);
```

Przykład naprawiony

```
element.textContent = location.hash.substring(1);
// lub: element.innerHTML = DOMPurify.sanitize(userInput);
```

Rekomendacja

Używaj `textContent` zamiast `innerHTML`; sanityzuj (DOMPurify); włącz auto-escaping w szablonach; w Java użyj OWASP Encoder; w C# `HtmlEncode`.

Katalog reguł — CSRF (CWE-352)

Plik reguł: `app/services/security_scan/rules/csrf_rules.py` OWASP: A01:2021-Broken Access Control · CWE: CWE-352

Opis klasy podatności

Cross-Site Request Forgery: atakujący zmusza uwierzytelnionego użytkownika do wykonania niezamierzonej operacji zmieniającej stan. Detekcja liniowa nie może udowodnić braku tokena CSRF w całym formularzu/handlerze, więc reguły celują w **jawne wyłączenie lub obejście** ochrony CSRF (sygnały wysokiej pewności).

Reguły

PY-CSRF-DISABLED (Python, high, conf 0.7)

```
@csrf_exempt|csrf_exempt\s*\
(|WTF_CSRF_ENABLED\s*=\s*False|CSRF_ENABLED\s*=\s*False
```

JAVA-CSRF-DISABLED (Java, high, conf 0.75)

```
\.csrf\s*\(\s*\)\s*\.\s*disable\s*\(|csrf\(\)\.ignoringAntMatchers
```

JS-CSRF-DISABLED (JS/TS, medium, conf 0.55)

```
csrf\s*:\s*false|ignoreMethods\s*:\s*\[[^\]]*POST
```

Przykład podatny (Django)

```
@csrf_exempt
def transfer_funds(request):
    ...
```

Przykład naprawiony

```
def transfer_funds(request):  # ochrona CSRF aktywna (bez @csrf_exempt)
    ...
```

Rekomendacja

Nie wyłączaj CSRF dla endpointów zmieniających stan. W Spring pozostaw ochronę CSRF; w Express włącz middleware CSRF i dołączaj token do POST/PUT/DELETE.

Katalog reguł — Broken Authorization / IDOR (CWE-862/639)

Plik reguł: `app/services/security_scan/rules/authz_rules.py` OWASP: A01:2021-Broken Access Control

Opis klasy podatności

Broken Access Control obejmuje brak kontroli autoryzacji (CWE-862) oraz IDOR — Insecure Direct Object Reference (CWE-639), gdy obiekt jest pobierany po identyfikatorze z requestu bez sprawdzenia właściciela. Autoryzacja jest kontekstowa, więc reguły celują w jawne markery obejścia i bezpośrednie odczyty obiektów kluczowane inputem.

Reguły

PY-IDOR-LOOKUP (Python, idor, CWE-639, high, conf 0.5)

```
objects\.(get|filter)\s*\([^)]*=\s*request\.|get_object_or_404\s*\
([^)]*request\.|\.query\.get\s*\(\s*request\.
```

PY-AUTH-BYPASS (Python, broken_authz, CWE-862, high, conf 0.55)

```
permission_classes\s*=\s*\[\s*AllowAny|authentication_classes\s*=\s*\
[\s*\]
```

JAVA-IDOR-LOOKUP (Java, idor, CWE-639, high, conf 0.45)

```
\.findById\s*\(\s*(request|@PathVariable|id\s*\))
```

JS-AUTH-BYPASS (JS/TS, broken_authz, CWE-862, high, conf 0.4)

```
//\s*(TODO|FIXME)[^\n]*\bauth|requireAuth\s*:\s*false
```

Przykład podatny (Django, IDOR)

```
invoice = Invoice.objects.get(id=request.GET["invoice_id"])
```

Przykład naprawiony

```
invoice = Invoice.objects.get(pk=invoice_id)
if invoice.owner_id != request.user.id:
    raise PermissionDenied
```

Rekomendacja

Weryfikuj własność/uprawnienia (np. `filter(owner=current_user)`); stosuj zasadę domyślnego odrzucania; wymagaj uwierzytelnienia przed dostępem do zasobu.

Katalog reguł — walidacja wejścia i injection

Plik reguł: `app/services/security_scan/rules/input_validation_rules.py`

Grupuje rodzinę „niebezpieczna operacja na niezaufanym wejściu”: command injection, code injection, path traversal, insecure deserialization i weak cryptography.

Reguły

PY-OS-SYSTEM (Python, command_injection, CWE-78, critical, conf 0.75)

```
os\.system\s*\(|subprocess\.[a-z_]+\([^)]*shell\s*=\s*True
```

JS-CHILD-PROCESS (JS/TS, command_injection, CWE-78, critical, conf 0.7)

```
child_process\.exec\s*\(|\bexec\s*\(\s*[`"'].*\$\{
```

PY-EVAL-EXEC (Python, code_injection, CWE-95, high, conf 0.7)

```
\b(eval|exec)\s*\(
```

JS-EVAL (JS/TS, code_injection, CWE-95, high, conf 0.7)

```
\beval\s*\(|new\s+Function\s*\(
```

PATH-TRAVERSAL (wszystkie języki, path_traversal, CWE-22, medium, conf 0.5)

```
open\s*\(\s*[^)]*\+|os\.path\.join\s*\([^)]*request|new\s+File\s*\
([^)]*\+
```

PY-PICKLE (Python, insecure_deserialization, CWE-502, high, conf 0.65)

```
pickle\.loads?\s*\(|yaml\.load\s*\((?![^)]*Loader)
```

WEAK-HASH (wszystkie języki, weak_cryptography, CWE-327, medium, conf 0.6)

```
\b(md5|sha1)\s*\(|hashlib\.(md5|sha1)\b|MessageDigest\.getInstance\(\s*["']](MD5|SHA-1)
```

Przykład podatny (command injection)

```
os.system("ping -c 1 " + user_host)
```

Przykład naprawiony

```
subprocess.run(["ping", "-c", "1", user_host], shell=False)
```

Rekomendacje skrótem

- Command injection: przekazuj argumenty listą, bez `shell=True`; użyj `execFile` / `spawn`.
- Code injection: unikaj `eval` / `exec`; użyj `ast.literal_eval` / `JSON.parse`.
- Path traversal: waliduj i normalizuj ścieżki; ogranicz do katalogu bazowego.
- Deserializacja: `json` lub `yaml.safe_load`.
- Krypto: SHA-256+ lub bcrypt/argon2 dla haseł.

Katalog reguł — Hardcoded Secrets (CWE-798)

Plik reguł: `app/services/security_scan/rules/secrets_rules.py` OWASP: A07:2021-Identification and Authentication Failures (część — A02 dla klucza prywatnego)

Opis klasy podatności

Zakodowane na stałe sekrety (hasła, klucze API, tokeny, klucze prywatne) w kodzie źródłowym są łatwe do wykrycia w repozytorium i prowadzą do przejęcia dostępu.

Reguły

SECRET-HARDCODED (wszystkie języki, high, conf 0.55)

```
(password|passwd|pwd|secret|api[_-]?key|apikey|token|access[_-]?key)\s*[:=]\s*["'][^"'\s]{6,}["']
```

Sekret przypisany literałem ≥6 znaków.

SECRET-PRIVATE-KEY (wszystkie języki, critical, conf 0.9)

```
-----BEGIN (RSA |EC |DSA |OPENSSH |PGP )?PRIVATE KEY-----
```

SECRET-AWS-KEY (wszystkie języki, critical, conf 0.85)

```
\bAKIA[0-9A-Z]{16}\b
```

SECRET-JWT (wszystkie języki, high, conf 0.7)

```
\beyJ[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]+
```

Przykład podatny

```
DB_PASSWORD = "Sup3rS3cr3tP@ss"
```

Przykład naprawiony

```
DB_PASSWORD = os.environ["DB_PASSWORD"]
```

Rekomendacja

Przenieś sekrety do zmiennych środowiskowych / menedżera sekretów. Wykryty sekret należy **usunąć z repozytorium i zrotować** (sama zmiana nie wystarczy — wartość pozostaje w historii git).

Katalog reguł — macierz języków

Stałe językowe (`rules/base.py`):

- `PY` = {python}
- `JS` = {javascript, typescript}
- `JAVA` = {java}
- `CS` = {csharp, c#}
- `C` = {c, cpp, c++}
- `ALL` = frozenset() (pusty zbiór = wszystkie języki)

Funkcja `rx(pattern)` kompiluje regex z flagą `re.IGNORECASE`.

Macierz reguła → języki

rule_id	typ	CWE	Python	JS/TS	Java	C#	Wszystkie
SQLI-STRING-CONCAT	sql_injection	89					✓
PY-CURSOR-FSTRING	sql_injection	89	✓				
JAVA-STATEMENT-CONCAT	sql_injection	89			✓		
JS-QUERY-CONCAT	sql_injection	89		✓			
CS-SQLCOMMAND-CONCAT	sql_injection	89				✓	
JS-DOM-SINK	xss	79		✓			
JS-FRAMEWORK-RAW-HTML	xss	79		✓			
PY-TEMPLATE-RAW	xss	79	✓				
JAVA-RESPONSE-WRITE	xss	79			✓		
CS-RESPONSE-RAW	xss	79				✓	
PY-CSRF-DISABLED	csrf	352	✓				
JAVA-CSRF-DISABLED	csrf	352			✓		
JS-CSRF-DISABLED	csrf	352		✓			

rule_id	typ	CWE	Python	JS/TS	Java	C#	Wszystkie
PY-IDOR-LOOKUP	idor	639	✓				
PY-AUTH-BYPASS	broken_authz	862	✓				
JAVA-IDOR-LOOKUP	idor	639			✓		
JS-AUTH-BYPASS	broken_authz	862		✓			
PY-OS-SYSTEM	command_injection	78	✓				
JS-CHILD-PROCESS	command_injection	78		✓			
PY-EVAL-EXEC	code_injection	95	✓				
JS-EVAL	code_injection	95		✓			
PATH-TRAVERSAL	path_traversal	22					✓
PY-PICKLE	insecure_deserialization	502	✓				
WEAK-HASH	weak_cryptography	327					✓
SECRET-HARDCODED	hardcoded_secret	798					✓
SECRET-PRIVATE-KEY	hardcoded_secret	798					✓
SECRET-AWS-KEY	hardcoded_secret	798					✓
SECRET-JWT	hardcoded_secret	798					✓

Podsumowanie pokrycia

- **26 reguł** w 6 kategoriach (SQLi 5, XSS 5, CSRF 3, authz 4, input validation 7, secrets 4 — łącznie z PATH/PICKLE/WEAK-HASH liczonymi w input validation).
- **12 CWE:** 20, 22, 78, 79, 89, 95, 327, 352, 502, 639, 798, 862.
- **OWASP 2021:** A01 (6 reguł), A02 (2), A03 (14), A07 (3), A08 (1).

Rejestr reguł (`rules/registry.py`)

Kolejność agregacji `ALL_RULES`: SQLI → XSS → CSRF → AUTHZ → INPUT_VALIDATION → SECRETS. `RuleRegistry` udostępnia `get_rules(language)`, `get_rule_by_id(id)`, `get_rules_by_cwe(cwe)`. Singleton przez `get_registry()`; kompatybilność: `rules_for_language(language)`.

Katalog reguł — definicje taint (sources / sinks / sanitizers)

Pliki: `app/services/security_scan/taint_definitions/`

Struktury bazowe (`base.py`)

- **TaintSource** — `name`, `patterns`, `language`, `taint_type` ∈ {user_input, file_input, env_input}.
- **TaintSink** — `name`, `patterns`, `language`, `sink_type` ∈ {sql, html, exec, file, log}, `vulnerability_type`, `cwe_id`, `severity`.
- **TaintSanitizer** — `name`, `patterns`, `language`, `protects_against` (frozenset typów sinków), metoda `protects(sink_type)`.

Domyślna severity sinka (`SINK_SEVERITY`): exec→critical, sql→high, html→medium, file→medium, log→low.

Python

Kategoria	Pozycje
Sources	`request.(GET|POST|args|form|...)`, `input()`, `sys.argv`, `os.environ` / `os.getenv`, `.read()` / `.readline()`
Sinks	`.execute()` / `.executemany()` (sql), `os.system` / `subprocess.*` (exec), `eval` / `exec` (exec), `render_template_string` / `Markup` (html), `open()` (file)
Sanitizers	`html.escape` / `escape` →{html}, `bleach.clean` →{html}, `int` / `float` →{sql,exec,file}, `shlex.quote` →{exec}

JavaScript / TypeScript

Kategoria	Pozycje
Sources	`req.(body|params|query|...)`, `location.(hash|search|href)` / `document.cookie` / `window.name`, `process.env.`
Sinks	`.query` / `.execute` (sql), `.innerHTML=` / `.outerHTML=` / `document.write` (html), `eval` / `new Function` (exec), `child_process.exec` / `exec` (exec)
Sanitizers	`DOMPurify.sanitize` →{html}, `escapeHtml` / `escape` →{html}, `parseInt` / `Number` →{sql,exec,html}

Java

Kategoria	Pozycje
Sources	`request.getParameter` / `getHeader` / `getQueryString`, `@RequestParam` / `@PathVariable`, `System.getenv`

Kategoria	Pozycje
Sinks	`.execute(Query|Update)?` / `createStatement` (sql), `Runtime.getRuntime().exec` / `new ProcessBuilder` (exec), `.getWriter().(print|write)` / `.println` (html)
Sanitizers	`prepareStatement` / `setString` →{sql}, `ESAPI.encoder()` / `Encode.forHtml` →{html}, `Integer.parseInt` / `Long.parseLong` →{sql,exec,html}

C#

Kategoria	Pozycje
Sources	`Request.(Form|QueryString|...)` / `Request["..."]`, `ModelState`, `Environment.GetEnvironmentVariable`
Sinks	`new SqlCommand` / `.ExecuteReader` / `.ExecuteNonQuery` (sql), `Process.Start` (exec), `Response.Write` (html)
Sanitizers	`new SqlParameter` / `.Parameters.Add` →{sql}, `HtmlEncode` / `HttpUtility.HtmlEncode` →{html}, `int.Parse` / `Convert.ToInt32` →{sql,exec,html}

C / C++

Kategoria	Pozycje
Sources	`argv[` / `argc`, `fgets` / `scanf` / `gets`, `getenv`
Sinks	`system` / `popen` / `execvp?` (exec), `sprintf` / `strcpy` / `strcat` (buffer_overflow, CWE-120), `fopen` / `open` (file)
Sanitizers	`snprintf` / `strncpy` / `strncat` →{exec}, `atoi` / `strtol` →{exec,file}

Adekwatność sanitizera

Sanitizer chroni wyłącznie przed sinkami z `protects_against`. Np. `html.escape` NIE chroni przed sinkiem `sql` — taka ścieżka jest flagowana z `sanitizer_adequate = False` (patrz taint analyzer).

Pipeline DAST — integracja OWASP ZAP

Plik: `app/services/security_scan/dast_runner.py`

1. Architektura integracji

`DASTRunner` steruje instancją OWASP ZAP (obraz `owasp/zap2docker-stable`) przez protokół `ZAPController` (Protocol), który abstrahuje kontener Docker i REST API ZAP. Dzięki temu w testach można wstrzyknąć atrapę kontrolera bez uruchamiania prawdziwego ZAP.

2. Cykl życia skanu

```
start kontenera → oczekiwanie na gotowość → (autentykacja) →
spider → active scan → polling statusu → pobranie alertów →
sanityzacja → mapowanie na SecurityFinding → zatrzymanie
```

3. Konfiguracja

Parametr	Wartość domyślna	Znaczenie
`_DEFAULT_SCAN_TIMEOUT`	1800 s (30 min)	maks. czas active scan
`_DEFAULT_READY_TIMEOUT`	120 s (2 min)	oczekiwanie na start ZAP

Gotowość sprawdzana przez polling endpointu `/JSON/core/view/version/`.

4. Wywołanie

```
async def run_dast(project_id, tenant_id, scan_id, target_url,
         auth_config) -> list[SecurityFinding]
```

`target_url` jest wymagany (walidacja na wejściu). `auth_config` opcjonalny — patrz autentykacja.

5. Endpoint API

Skan DAST uruchamiany asynchronicznie przez `POST /projects/{project_id}/security-scan` z `scan_type=DAST`, `target_url` i opcjonalnym `auth_config`. Dyspozytor wywołuje `run_dast_scan.delay(scan_id, project_id, tenant_id, target_url, auth_config_dict)`.

6. Limity i bezpieczeństwo

- Skan tylko na autoryzowanych celach (target uruchomionej aplikacji projektu).
- Request/response w znaleziskach są sanityzowane (patrz parsowanie wyników).

Pipeline DAST — payloady i fuzzing

1. Mechanizm aktywnego skanu

Aktywny skan ZAP (active scan) automatycznie wstrzykuje payloady ataków do wykrytych punktów wejścia (parametry GET/POST, nagłówki, ciasteczka), zidentyfikowanych wcześniej w fazie spider (crawl). TestGEN korzysta z wbudowanego zestawu reguł aktywnego skanowania ZAP.

2. Typy ataków wykrywanych dynamicznie

Aktywny skan ZAP pokrywa m.in.:

- SQL Injection (plugin 40018 → CWE-89)
- Reflected XSS (plugin 40012 → CWE-79)
- OS Command Injection (plugin 90020 → CWE-78)
- Path Traversal, Remote File Inclusion
- nagłówki bezpieczeństwa, ujawnienie informacji

Mapowanie plugin ZAP → CWE realizuje `_PLUGIN_TO_CWE` w `dast_runner.py` (fallback, gdy alert nie zawiera własnego CWE).

3. Limity wydajności

Limit	Wartość
Timeout active scan	1800 s (30 min)
Timeout gotowości	120 s

Po przekroczeniu limitu skanu runner przerywa polling i przetwarza alerty zebrane do tej pory, zamiast czekać w nieskończoność.

4. Faza spider

Przed aktywnym skanem uruchamiana jest faza spider, która odkrywa powierzchnię ataku (URL-e, formularze). Tylko odkryte punkty są następnie poddawane fuzzingowi w aktywnym skanie, co ogranicza zakres do rzeczywistej aplikacji.

5. Bezpieczeństwo payloadów

Payloady aktywnego skanu mają charakter wykrywający (detekcja podatności), nie destrukcyjny. Skan należy uruchamiać wyłącznie na środowiskach testowych / autoryzowanych celach projektu.

Pipeline DAST — autentykacja

Struktura: `AuthConfig` (`dast_runner.py`)

1. Obsługiwane metody

`AuthConfig.auth_type` ∈ {`form`, `token`, `basic`}. Fabryka `AuthConfig.from_dict()` waliduje typ i ekstrahuje wymagane pola z `credentials`.

Metoda	Wymagane pola credentials
form	`login_url`, `username_field`, `password_field`, `username`, `password`
token	`token`
basic	`username`, `password`

2. Konfiguracja w ZAP

Autentykacja konfigurowana jest przez ZAP Replacer API (wstrzykiwanie nagłówków/wartości) zanim uruchomiony zostanie spider i active scan, dzięki czemu skaner działa jako uwierzytelniony użytkownik i pokrywa chronione ścieżki aplikacji.

3. Przykład (token)

```
{
  "auth_type": "token",
  "credentials": { "token": "Bearer eyJ..." }
}
```

4. Przykład (form)

```
{
  "auth_type": "form",
  "credentials": {
    "login_url": "https://app.example/login",
    "username_field": "email",
    "password_field": "password",
    "username": "tester@example.com",
    "password": "***"
  }
}
```

5. Testy kontroli dostępu

Uwierzytelniony skan umożliwia wykrywanie podatności dostępnych tylko po zalogowaniu (np. IDOR na zasobach użytkownika). Dane uwierzytelniające nie są zapisywane w znaleziskach — request/response są sanityzowane (nagłówki

`authorization`, `cookie`, tokeny → `[REDACTED]`).

Pipeline DAST — parsowanie wyników

1. Od alertu ZAP do SecurityFinding

```
alerty ZAP (JSON) → _parse_zap_alerts → DASTFinding →
_sanitize_request_response → _to_finding → SecurityFinding
```

2. DASTFinding

Pola: `url`, `method`, `vulnerability_type`, `severity`, `cwe_id`, `evidence` (fragment odpowiedzi), `plugin_id`, `request`, `response` (sanityzowane), `recommendation`, `extra`.

3. Mapowanie severity

Ryzyko ZAP → `Severity`:

Ryzyko ZAP	Severity
high	HIGH
medium	MEDIUM
low	LOW
informational / info	INFO

`_map_zap_risk_to_severity()` realizuje to mapowanie.

4. Mapowanie CWE

`_map_zap_to_cwe(plugin_id, cweid)`:

1. Jeśli alert ma własny `cweid` — użyj go.
2. W przeciwnym razie fallback `_PLUGIN_TO_CWE` (np. 40018→CWE-89, 40012→CWE-79, 90020→CWE-78).

5. Sanityzacja request/response

`_sanitize_request_response(request, response)`:

- Nagłówki wrażliwe → `[REDACTED]`: `authorization`, `cookie`, `set-cookie`, `x-api-key`, `x-auth-token`, `proxy-authorization`.
- `_TOKEN_RE` zastępuje wartości po `password` / `passwd` / `token` / `secret` / `api_key` / `access_token` → `[REDACTED]`.

Zapewnia to, że dane uwierzytelniające i tokeny nie trafiają do bazy znalezisk ani raportów.

6. Korelacja z SAST

Znaleziska DAST są następnie korelowane ze znaleziskami SAST przez `FindingAggregator` (boost +0.20 dla pary SAST↔DAST) — dynamiczne potwierdzenie podatności statycznej podnosi zaufanie i może eskalować severity w risk engine.

Pipeline CVE — źródła danych

Plik: `app/services/security_scan/cve_pipeline.py` Klasa publiczna: `CVEPipeline`

1. Cel

Pipeline CVE okresowo zasila lokalną tabelę `cve_database` danymi z trzech publicznych źródeł, dzięki czemu dependency scanner może działać offline z aktualną wiedzą o podatnościach.

2. Źródła danych

Źródło	URL	Metoda
NVD (NIST)	`https://services.nvd.nist.gov/rest/json/cves/2.0`	REST v2.0, opcjonalny `nvd_api_key`
OSV.dev	`https://api.osv.dev/v1/query`	query per ekosystem
GitHub Advisory	`https://api.github.com/advisories`	bearer `github_token`

Ekosystemy odpytywane w OSV: `PyPI`, `npm`, `Maven`, `NuGet`, `Packagist`.

3. Parametry NVD

`pubStartDate`, `pubEndDate`, `resultsPerPage=2000`, `startIndex` (paginacja). Nagłówek z kluczem API opcjonalny (podnosi limity rate).

4. Struktury danych

- **CVERecord** — `cve_id`, `package_name`, `ecosystem`, `cvss_score`, `description`, `severity`, `affected_versions`, `fixed_version`, `published_date`, `source_db`. Metoda `key() = (cve_id, package_name)`.
- **SourceStatus** — `source`, `ok`, `fetched`, `error`.
- **CVEUpdateResult** — `total_new`, `total_updated`, `affected_projects`, `sources_status`, `errors`.

5. Metody pobierania

```
async def _fetch_nvd_updates(since) -> list[CVERecord]
async def _fetch_osv_updates(since) -> list[CVERecord]
async def _fetch_github_advisories(since) -> list[CVERecord]
```

Pobieranie z trzech źródeł odbywa się równolegle. Parsery (`_parse_nvd`, `_parse_osv`, `_parse_github`) są czyste/statyczne.

6. Deduplikacja i scalanie

`_merge_and_deduplicate(by_source)` — klucz `(cve_id, package_name)`, priorytet: wyższy CVSS wygrywa; kolejność źródeł NVD > OSV > GitHub.

7. Persistencja

`_save_to_database(records, db)` upsertuje do `CVEDatabase` i zwraca `(new_count, updated_count)`.

Pipeline CVE — harmonogram i konfiguracja

1. Harmonogram (Celery Beat)

Wpis w `celery_app.py`:

```
"cve-update-database-monthly": {
    "task": "security.update_cve_database",
    "schedule": crontab(hour=2, minute=0, day_of_month=1),
    "options": {"queue": "security"},
}
```

- Uruchomienie: **02:00 UTC, 1. dnia każdego miesiąca**.
- Nazwa zadania: `security.update_cve_database`.
- Kolejka: `security`.

2. Konfiguracja zadania (`cve_tasks.py`)

`update_cve_database()`:

Parametr	Wartość
Max retries	3
Hard time limit	3600 s (1 h)
Soft time limit	3300 s (55 min)
Retry backoff	10 s, 60 s, 300 s

Zadanie zapisuje `CVEUpdateLog` ze statusem (running → completed/failed).

3. Rate limiting

- NVD: opcjonalny `nvd_api_key` (podnosi limit zapytań).
- GitHub: `github_token` (bearer).
- Paginacja NVD: `resultsPerPage=2000` + `startIndex`.

Pobieranie z trzech źródeł równoległe, ale każde źródło respektuje własne limity API; błędy źródła są raportowane w `SourceStatus` i nie przerywają całego przebiegu.

4. Orkiestracja głównego przebiegu

```
async def update_cve_database(db) -> CVEUpdateResult
```

Kroki: fetch (NVD/OSV/GitHub równolegle) → merge & deduplicate → persist → znajdź dotknięte projekty → wyzwól re-skany → wyślij powiadomienia. Szczegóły re-skanu: auto-rescan.

5. Walidacja na danych golden

Auto-generowane reguły są walidowane na korpusie golden przed włączeniem — patrz auto-generowanie reguł. Próg fałszywych dopozytywów: `_MAX_FPR = 0.05` (5%).

Pipeline CVE — auto-generowanie reguł SAST

1. Cel

Dla wybranych nowych rekordów CVE pipeline może wygenerować szkic reguły SAST (regex + typ + CWE), aby wykrywać podatny wzorzec bezpośrednio w kodzie — uzupełnienie detekcji opartej na wersji pakietu.

2. Generowanie

```
async def _generate_sast_rules(records) -> list[VulnerabilityRule]
```

Na podstawie opisu CVE powstaje szkic reguły z polami `REGEX:` / `TYPE:` / `CWE:` (`module="security"`, `task_type="cve_rule_generation"`).

3. Parsowanie szkicu

```
def _parse_rule_response(text, rec) -> VulnerabilityRule | None
```

Ekstrahuje pola `REGEX:` / `TYPE:` / `CWE:` i tworzy `VulnerabilityRule` z `confidence_weight=0.3` (niższe zaufanie niż reguły ręczne — szkic wymaga potwierdzenia w praktyce).

4. Walidacja na golden data

```
def _validate_rule_on_golden_data(rule, safe_samples) -> RuleValidation
```

- Oblicza FPR (false-positive rate) na próbkach bezpiecznych.
- Próg: `_MAX_FPR = 0.05` (5%).
- `RuleValidation` — `rule_id`, `fpr`, `passed`, `sample_size`.

Reguła, która generuje zbyt wiele fałszywych alarmów (FPR ≥ 5%), nie przechodzi walidacji i nie jest włączana. Powiązanie z korpusem golden: golden test data.

5. Bezpieczeństwo procesu

Auto-generowanie jest bramkowane walidacją FPR na golden data, co chroni przed degradacją jakości detekcji przy cichym dodawaniu reguł. To uzupełnia regresyjną suitę progową ze Story 4.14.

Pipeline CVE — auto re-skan projektów i powiadomienia

1. Wykrywanie dotkniętych projektów

```
async def _find_affected_projects(records, db) -> list[AffectedProject]
```

Po zapisaniu nowych CVE pipeline porównuje je z zależnościami projektów i buduje listę `AffectedProject` (`project_id`, `project_name`, `owner_id`, `affected_packages`, `new_cves`).

2. Wyzwolenie re-skanu

```
def _trigger_dependency_rescan(project_ids) -> None
```

Wywołuje zadanie Celery `rescan_affected_projects.delay(project_ids)`.

`rescan_affected_projects(project_ids)` (`cve_tasks.py`):

- Tworzy rekordy `SecurityScan` z `ScanType.DEPENDENCY`.
- Dyspozytuje `run_dependency_scan.delay()` per projekt.

Dzięki temu projekty z nowo dotkniętymi pakietami są automatycznie ponownie skanowane bez udziału użytkownika.

3. Powiadomienia

```
async def _send_notifications(affected, db) -> None
```

Loguje ustrukturyzowane powiadomienie (haki e-mail jako placeholder) dla właścicieli dotkniętych projektów.

4. Przepływ end-to-end

```
update_cve_database (cron 1. dnia, 02:00 UTC)
  → fetch NVD/OSV/GitHub
  → merge + deduplicate
  → save_to_database
  → find_affected_projects
  → trigger_dependency_rescan (Celery)
        → SecurityScan(DEPENDENCY) + run_dependency_scan
  → send_notifications
  → CVEUpdateResult (total_new, total_updated, affected_projects, errors)
```

5. Logowanie przebiegu

Każdy przebieg zapisuje `CVEUpdateLog` (running → completed/failed) oraz `SourceStatus` per źródło, co umożliwia audyt i diagnozę niepowodzeń pobierania.

Architektura modułu luk funkcjonalnych — przegląd

> Moduł luk funkcjonalnych TestGEN (Epic 5). Wersja odpowiada implementacji w `backend/app/services/gap_analysis/`.

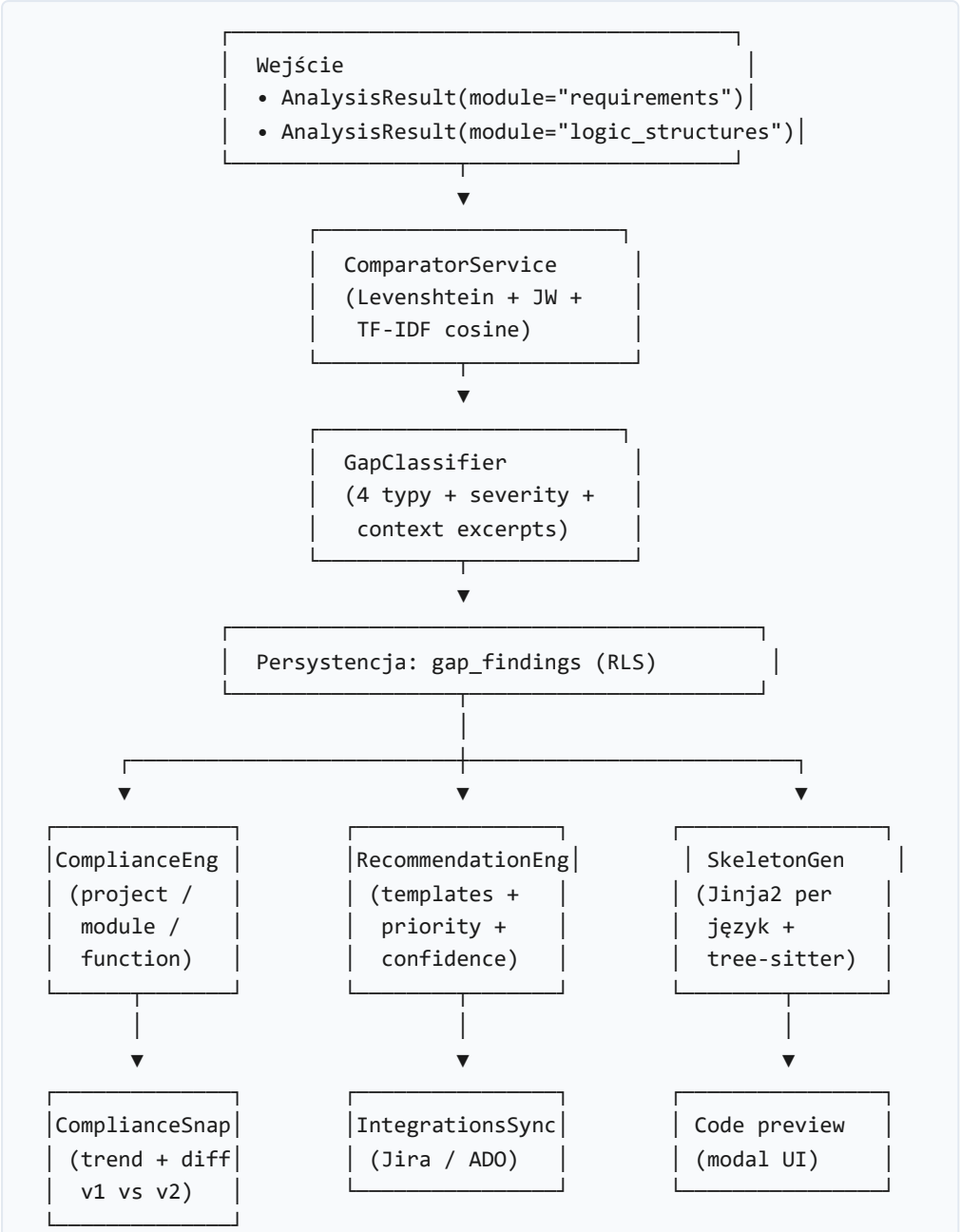
1. Cel modułu

Moduł luk funkcjonalnych porównuje wymagania z dokumentacji (SRS, SDD, user stories) z faktycznie zaimplementowanym kodem i produkuje ustrukturyzowaną listę luk wraz z oceną zgodności (compliance) oraz rekomendacjami naprawczymi.

Główne funkcje (FR38–FR42):

- **FR38** — Dopasowanie wymagań do bytów kodu (fuzzy + semantic matching)
- **FR39** — Klasyfikacja luk w 4 kategoriach: `missing`, `different`, `extra`, `inconsistent`
- **FR40** — Compliance scoring na poziomie projektu / modułu / funkcji
- **FR41** — Detekcja błędów kodu (syntax, logic, semantic, conformance)
- **FR42** — Rekomendacje + skeleton code generator + integracja z trackerami

2. Diagram pipeline'u



3. Moduły i ich odpowiedzialności

Moduł	Plik	Rola
Comparator	`comparator.py`	Fuzzy matching wymagań z bytami kodu (3 algorytmy)
Classifier	`classifier.py`	Klasyfikacja w 4 typy luk + przypisanie severity
Classification rules	`classification_rules.py`	Thresholds, exclude patterns, severity map

Moduł	Plik	Rola
Compliance engine	`compliance_engine.py`	Scoring 0–100 per projekt/moduł/funkcja
Error detectors	`error_detectors/`	Syntax / logic / semantic / conformance
Recommendation engine	`recommendation_engine.py`	Rekomendacje na podstawie reguł i szablonów
Skeleton generator	`skeleton_generator.py`	Jinja2 + tree-sitter (Python/TS/Java/C#)
Comparison engine	`comparison_engine.py`	Diff snapshotów v1↔v2 + trend
Integrations	`integrations/jira_service.py`, `integrations/azure_devops_service.py`	Sync do trackerów

4. Tryb pracy

Analiza działa na deterministycznych algorytmach (Levenshtein, Jaro-Winkler, TF-IDF cosine): offline, bez zależności od usług zewnętrznych, z gwarancją powtarzalności wyników — te same dane wejściowe dają ten sam wynik.

5. Model danych

Wszystkie znaleziska są persystowane w dedykowanej tabeli `gap_findings` (wzorzec Epic 4) z RLS per-tenant. Pozwala na niezależne filtrowanie, paginację i zapytania cross-analysis, których nie obsługuje JSONB w `AnalysisResult.result_data`.

Powiązane tabele:

- `gap_findings` — znaleziska luk (4 typy + source + match_score)
- `code_errors` — błędy kodu (4 kategorie)
- `recommendations` — rekomendacje naprawcze z confidence i priority
- `compliance_snapshots` — punkt w czasie dla porównań wersji
- `integration_configs` + `integration_sync_logs` — konfiguracja i logi sync do trackerów

Szczegóły: 09 — Modele danych.

6. Przepustowość i ograniczenia

- Comparator: O(R×E) par dopasowania (R = wymagania, E = byty kodu). Dla typowych projektów (dziesiątki do niskich setek wymagań) < 1 s.
- Skeleton generator: tree-sitter waliduje składnię; 1 plik na finding type=missing.

7. Powiązane dokumenty

- 02 — Comparator
- 03 — Klasyfikator luk
- 04 — Compliance engine
- 05 — Recommendation engine
- 06 — Skeleton generator
- 08 — Integracje
- 09 — Modele danych

Comparator — fuzzy matching wymagań z kodem

> Plik: `backend/app/services/gap_analysis/comparator.py`

1. Odpowiedzialność

`ComparatorService` przyjmuje listę wymagań (`RequirementData`) i listę bytów kodowych (`CodeEntity`) i dla każdej pary oblicza stopień podobieństwa. Wynik przekazuje do `GapClassifier`, który podejmuje decyzję o typie luki.

2. Algorytmy dopasowania

Trzy komplementarne algorytmy czysto-Pythonowe (bez zewnętrznych zależności): scikit-learn / jellyfish / thefuzz są celowo niewykorzystywane — dla projektów o wielkości dziesiątek do niskich setek wymagań wydajność czystego Pythona jest wystarczająca, a binary deps pozostają minimalne.

2.1 Levenshtein (edytowanie odległości)

`_levenshtein_match(req_name, code_name)` — odległość edycyjna znormalizowana do [0.0, 100.0]:

```
score = (1 - dist / max_len) * 100
```

Klasyczny algorytm DP (m×n), ~25 linii. Dobra detekcja literówek i drobnych zmian nazewnictwa (`getUserData` ↔ `get_user_data`).

2.2 Jaro-Winkler

`_jaro_winkler_match(req_name, code_name)` — Jaro similarity z premią dla wspólnego prefiksu (Winkler). ~30 linii. Preferowany dla nazw z melodijnym prefiksem (`authenticate` ↔ `authn`).

Wzór:

```
J = (1/3) * (m/|s1| + m/|s2| + (m-t)/m)        # Jaro
JW = J + l * p * (1 - J)                          # Winkler, l ≤ 4, p = 0.1
```

2.3 TF-IDF + cosine similarity

`_tfidf_cosine_match(req_text, code_text)` — semantyczne podobieństwo tekstu wymagania i nazwy + docstring bytu. Implementacja oparta na `collections.Counter` (TF) i IDF obliczanym w locie. ~40 linii.

Kroki:

1. Tokenizacja (lowercase + regex `\w+`)
2. Oblicz TF dla obu tekstów
3. IDF dla korpusu = oba teksty
4. Wektory TF-IDF
5. Cosine similarity

3. Kombinacja wyników

`_combine_scores(lev, jw, sem) -> float` — ważona średnia:

```
combined = lev * 0.25 + jw * 0.35 + sem * 0.40
```

Rationale wag:

- **Levenshtein 0.25** — najsłabszy dla różnic językowych, ale dobry na literówki.
- **Jaro-Winkler 0.35** — lepszy dla nazw funkcji (prefiksy).
- **Semantic TF-IDF 0.40** — najważniejszy; wykorzystuje pełny tekst wymagania i docstring.

Wagi są w stałej `_WEIGHTS` i mogą być strojone bez zmiany algorytmów.

4. Progi klasyfikacji

Po obliczeniu `combined` dla każdej pary, Comparator wybiera najlepsze dopasowanie per wymaganie i przekazuje do klasyfikatora:

Wynik combined	Klasyfikacja
`< 30.0`	brak dopasowania → `missing`
`30.0 – 70.0`	częściowe dopasowanie → `different`
`≥ 70.0`	dobre dopasowanie → wymaganie zaimplementowane (nie jest luką)

Progi są w `classification_rules.py`:

- `MISSING_THRESHOLD = 30.0`
- `DIVERGENT_THRESHOLD = 70.0`
- `GOOD_MATCH_THRESHOLD = 70.0`

5. Struktury danych

`RequirementData`

```
@dataclass
class RequirementData:
    req_id: str
    text: str
    category: str = ""
    priority: str = "medium"   # low | medium | high | critical
```

`CodeEntity` (z `services/analysis/gap_analyzer.py`)

```
@dataclass
class CodeEntity:
    entity_id: str
    name: str
    type: str          # function | method | class | ...
    file_path: str
    line_number: int
    language: str      # python | typescript | java | csharp
    docstring: str | None = None
```

`GapFindingData`

Wynik Comparator+Classifier. Pola: `requirement_id`, `code_entity_id`, `gap_type`, `description`, `requirement_text`, `code_reference`, `match_score`, `match_method`, `context`, `severity`.

6. API

```
service = ComparatorService()
findings, summary = service.compare(requirements, code_entities)
# findings: list[GapFindingData]
# summary:  GapAnalysisSummaryData
```

7. Testowanie

Testy jednostkowe: `backend/tests/unit/services/gap_analysis/test_comparator.py`. Golden testy (12 par kod+dokumentacja, 4 języki): `backend/tests/golden_data/gap_analysis/*.json` + runner `backend/tests/golden/test_gap_analysis_golden.py`.

AC golden:

- Recall ≥ 75% (per-case i średnia)
- Compliance accuracy ≥ 80% (missing count ±1)

8. Powiązane dokumenty

- 03 — Klasyfikator luk

Klasyfikator 4 typów luk

> Plik: `backend/app/services/gap_analysis/classifier.py` + `classification_rules.py`

1. Odpowiedzialność

`GapClassifier` bierze wynik dopasowania z `ComparatorService` i dla każdego wymagania produkuje ustrukturyzowane `GapFindingData` z:

- **gap_type** — jeden z 4 typów: `missing`, `different`, `extra`, `inconsistent`
- **severity** — `info` | `warning` | `critical`
- **context** — excerpt wymagania + excerpt bytu kodu (file/line)
- **description** — opis czytelny dla człowieka

2. Cztery typy luk

2.1 `missing` — brak implementacji

Wymaganie istnieje w dokumentacji, ale kod nie zawiera odpowiadającego bytu (lub score < `MISSING_THRESHOLD` = 30.0).

Przykład: wymagane "System must export to PDF", w kodzie brak jakiegokolwiek bytu o nazwie lub docstringu z "export"/"pdf".

Severity: zależna od priorytetu wymagań (patrz §4).

2.2 `different` — implementacja rozbieżna

Wymaganie jest częściowo zaimplementowane (score 30–70). Nazwa/docstring wskazują na pokrewieństwo, ale nie ma pewności, że kod spełnia wymaganie w pełni.

Przykład: wymaganie "must retry failed tasks with exponential backoff", kod ma `retryFailedTask()` ale bez widocznego mechanizmu backoff.

Severity: `warning` (domyślnie).

2.3 `extra` — kod bez wymagania

Byt kodu nie jest mapowany do żadnego wymagania. Traktowany jako potencjalny dead code lub implementacja poza zakresem.

Wykluczenia (regex `EXCLUDE_PATTERNS`): `test_*`, `__init__`, `conftest`, `setup`, `teardown`, `main`, `pytest_*`, `setUp`, `tearDown`, `before`, `after`.

Dodatkowo pomijane drobne akcesory (regex `MINOR_ENTITY_PATTERNS`): `get_*`, `set_*`, `is_*`, `has_*`, `can_*` — te zazwyczaj są pomocnicze, nie stanowią luky.

Severity: `info`.

2.4 `inconsistent` — sprzeczność między wymaganiami

Ten sam byt kodu jest najlepszym dopasowaniem dla 2+ wymagań, ale sygnały są sprzeczne. Detekcja oparta na:

1. **Score spread** > `INCONSISTENT_SCORE_SPREAD` = 40.0 — duże różnice w dopasowaniu między wymaganiami wskazują, że kod częściowo spełnia jedno, a ignoruje drugie.
2. **Contradiction indicators** — pary słów (`must` ↔ `must not`, `allow` ↔ `deny`, `enable` ↔ `disable`, `accept` ↔ `reject`, `include` ↔ `exclude`, `always` ↔ `never`, `required` ↔ `optional`) w wymaganiach mapowanych do tego samego bytu.

Severity: `critical` (zawsze).

3. Reguły klasyfikacji

```
# Pseudokod GapClassifier.classify()
for mapping in mappings:
    if mapping.best_score < MISSING_THRESHOLD:        # < 30
        classify_missing(mapping)
    elif mapping.best_score < DIVERGENT_THRESHOLD:    # 30-70
        classify_different(mapping)
    # else: ≥ 70 — dopasowane, nie jest luką

# Phase 3: detekcja inconsistent (cross-requirement)
for entity, req_ids in detect_inconsistent(mappings, all_scores):
    upgrade_or_create_inconsistent_findings(entity, req_ids)

# Phase 4: extra = byty kodu bez mapowania (po wykluczeniach)
for entity in code_entities:
    if entity.id not in matched_entity_ids and not is_excluded(entity):
        classify_extra(entity)
```

4. Mapowanie severity

Domyślne severity per gap_type (`SEVERITY_MAP`):

gap_type	severity
missing	warning
different	warning
extra	info
inconsistent	critical

Wyjątek dla `missing`: nadpisane przez priorytet wymagania (`PRIORITY_SEVERITY`):

priority	severity
high / critical	critical
medium / low	warning

Rationale: brak implementacji wymagań high-priority to ryzyko biznesowe, więc musi być traktowane jako critical.

5. Context excerpts

Każde finding zawiera `context` (JSONB) z:

- `requirement_excerpt` — pełny tekst wymagania
- `code_excerpt` — nazwa + docstring + file:line bytu
- `match_scores` — szczegółowe wyniki per algorytm (lev/jw/sem/combined)
- `alternative_matches` — kolejne 2 najlepsze kandydaty (jeśli > 1)

6. Konfiguracja

Wszystkie progi i reguły wydzielone do `classification_rules.py`:

```
MISSING_THRESHOLD = 30.0
DIVERGENT_THRESHOLD = 70.0
GOOD_MATCH_THRESHOLD = 70.0
INCONSISTENT_SCORE_SPREAD = 40.0

EXCLUDE_PATTERNS = re.compile(r"^(test_|__|conftest|...)$")
MINOR_ENTITY_PATTERNS = re.compile(r"^(get_|set_|is_|...)$")

SEVERITY_MAP = {"missing": "warning", "different": "warning", "extra":
         "info", "inconsistent": "critical"}
PRIORITY_SEVERITY = {"high": "critical", "medium": "warning", "low":
         "warning"}

CONTRADICTION_INDICATORS = [("must", "must not"), ("allow", "deny"), ...]
```

7. Testowanie

Testy jednostkowe:
`backend/tests/unit/services/gap_analysis/test_classifier.py` pokrywają wszystkie 4 typy, wykluczenia, priority severity upgrade, inconsistent detekcję.

8. Powiązane dokumenty

- 02 — Comparator
- 04 — Compliance engine

Compliance engine

> Plik: `backend/app/services/gap_analysis/compliance_engine.py`

1. Odpowiedzialność

`ComplianceEngine` oblicza zgodność implementacji z dokumentacją w skali 0–100, gdzie 100 = pełna zgodność (brak luk), 0 = maksymalna niezgodność.

Obliczenie jest **w locie** (compute-on-the-fly) z rekordów `gap_findings` — bez persystencji per finding. Snapshoty dla porównań wersji są persystowane osobno w `compliance_snapshots` (patrz Story 5.7).

2. Poziomy scoringu

Poziom	Funkcja	Grupowanie
Projekt	`calculate_project_score(findings)`	Globalnie
Moduł	`calculate_module_scores(findings)`	Per `file_path`
Funkcja	`calculate_function_scores(findings)`	Per `code_entity_id`

3. Wzór scoringu

Dla zbioru findings w danej grupie:

```
score = max(0.0, 100.0 - Σ penalty(f) for f in findings)
```

Penalty zależy od kombinacji gap_type i severity (`PENALTY_WEIGHTS`):

gap_type \ severity	critical	warning	info
missing	15	8	3
different	10	5	2
inconsistent	10	5	2
extra	5	3	1

Rationale:

- **Missing + critical** (15) — najdroższe; biznesowo ryzykowne.
- **Different + critical** (10) — częściowa implementacja krytycznego wymogu.
- **Inconsistent** (10/5/2) — sprzeczność wymagań; tyle samo co different.
- **Extra** (5/3/1) — kod poza zakresem dokumentacji; najmniej kosztowne.

Dolny limit 0 — żadna liczba luk nie zepchnie score poniżej zera.

4. Przykład

Projekt z 6 znaleziskami:

- 2× missing+critical = 2 × 15 = 30
- 1× missing+warning = 1 × 8 = 8
- 2× different+warning = 2 × 5 = 10
- 1× extra+info = 1 × 1 = 1

Penalty łączny: 49 → score = 100 − 49 = 51.0

5. Struktury wynikowe

```python
@dataclass
class ComplianceScore:
    score: float                        # 0–100
    total_findings: int
    breakdown: dict[str, int]           # {missing: 3, different: 2, ...}
    metadata: dict[str, Any]            # dodatkowe informacje

@dataclass
class ModuleScore:
    file_path: str
    score: float
    total_findings: int
    breakdown: dict[str, int]

@dataclass
class FunctionScore:
    code_entity_id: str
    score: float
    total_findings: int
    breakdown: dict[str, int]
    file_path: str | None
    entity_name: str | None
```

6. Endpointy API

- `GET /projects/{project_id}/compliance` — score projektu
- `GET /projects/{project_id}/compliance/modules` — lista score per moduł
- `GET /projects/{project_id}/compliance/functions` — lista score per funkcja
- `POST /projects/{project_id}/compliance/export` — CSV z wynikami

7. Eksport CSV

Eksport zawiera:

- wiersz nagłówka projektu
- wszystkie ModuleScores posortowane malejąco po liczbie findings
- wszystkie FunctionScores posortowane malejąco po liczbie findings

Format zgodny z Excel/LibreOffice (UTF-8 z separatorem przecinka).

8. Frontend

UI: `frontend/src/app/features/compliance/` — gauge wizualizujący score projektu, tabela modułów z drill-down do funkcji, eksport do CSV. Dostępny z Project Detail przyciskiem "Compliance".

9. Testowanie

Testy jednostkowe: `backend/tests/unit/services/gap_analysis/test_compliance_engine.py`. Testy E2E: `backend/tests/integration/test_gap_analysis_routes.py`.

10. Powiązane dokumenty

- 03 — Klasyfikator luk
- 01 — Przegląd

Recommendation engine

> Plik: `backend/app/services/gap_analysis/recommendation_engine.py`

1. Odpowiedzialność

`RecommendationEngine` produkuje actionable rekomendacje naprawcze dla każdej luki w `gap_findings`. Każda rekomendacja ma:

- **priority** — P1 (natychmiast) | P2 | P3 | P4 (niski)
- **confidence** — 0.0–1.0
- **suggested_action** — konkretna akcja (np. "Dodaj funkcję `export_pdf` w `src/analytics/export.py`")
- **effort_estimate** — szacowany wysiłek (S / M / L)

2. Mapowanie priorytetów

Priorytet zależy od gap_type i severity:

gap_type	severity	priority
missing	critical	**P1**
missing	warning	P2
missing	info	P3
different	critical	**P1**
different	warning	P2
inconsistent	(zawsze critical)	**P1**
extra	critical	P2
extra	warning	P3
extra	info	P4

Zasada: **P1 = natychmiastowej uwagi** (missing+critical, inconsistent, different+critical).

3. Rekomendacje szablonowe

Rekomendacje są deterministyczne, oparte na `gap_type` i kontekście.

Przykłady szablonów:

- **missing + critical**:

```
Zaimplementuj brakującą funkcjonalność: {requirement_text}.
Sugerowana lokalizacja: nowy moduł w {file_path} lub rozszerzenie
istniejącego.
```

- **different + warning**:

```
Przejrzyj implementację '{code_entity_name}' — wymaganie ({req_id})
wskazuje
na rozbieżność. Zweryfikuj, czy {requirement_excerpt} jest w pełni
spełnione.
```

- **extra + warning**:

```
Kod '{entity_name}' nie ma odpowiadającego wymagania w dokumentacji.
Rozważ: (a) dodanie wymagania, (b) oznaczenie jako @internal, (c)
usunięcie.
```

- **inconsistent**:

```
Sprzeczność: kod '{entity_name}' jest mapowany do 2+ wymagań z
przeciwnymi
sygnałami ({req_ids}). Rozdziel implementację lub ujednolić wymagania.
```

Confidence rekomendacji: 0.7 (wynik deterministyczny, z reguł).

5. Szacowanie wysiłku (effort_estimate)

Prosta heurystyka:

Warunek	Effort
missing + critical	L
missing + warning	M
missing + info	S
different	S
extra	S
inconsistent	M (wymaga koordynacji z PO)

6. Persystencja

Rekomendacje są persystowane w `recommendations` (model `backend/app/models/recommendation.py`) z:

- FK do `gap_findings.id`
- `priority`, `confidence`, `effort_estimate`
- `suggested_action` (Text)
- `source` (pochodzenie rekomendacji)

- `status` (open | accepted | applied | dismissed)

7. Endpointy API

- `GET /projects/{project_id}/recommendations` — lista z filtrami (priority, confidence_min, source, status)
- `POST /projects/{project_id}/recommendations/generate` — wymusza regenerację

8. Frontend

UI: panel "Rekomendacje" w widoku gap-analysis. Wyświetla listę z:

- priority badge (P1 czerwony, P2 pomarańczowy, P3 żółty, P4 niebieski)
- confidence jako procent + ostrzeżenie jeśli < 0.5
- suggested_action w karcie rozszerzalnej
- akcje: "Oznacz jako zaakceptowana" / "Generuj skeleton"

9. Testowanie

Testy jednostkowe: `backend/tests/unit/services/gap_analysis/test_recommendation_engine.py` pokrywają template-based generowanie, mapowanie priorytetów, edge case'y (brakujące pola).

10. Powiązane dokumenty

- 03 — Klasyfikator luk
- 06 — Skeleton generator

Skeleton generator

> Plik: `backend/app/services/gap_analysis/skeleton_generator.py`

1. Odpowiedzialność

`SkeletonGenerator` produkuje szkielet implementacji dla luk typu `missing` — czyli tam, gdzie wymaganie istnieje, ale kod nie zawiera odpowiadającego bytu.

Szkielet to gotowy do wklejenia plik źródłowy z:

- poprawną składnią języka
- sygnaturą funkcji / klasy opartą na nazwie wymagania
- docstringiem / komentarzem TODO wskazującym powiązane wymaganie
- stub implementacji (NotImplementedError / NotImplemented / zwracający null)

2. Obsługiwane języki

Język	Rozszerzenie	Skeleton pattern
Python	`.py`	`def func(...): raise NotImplementedError(...)`
TypeScript	`.ts`	`function func(...): T { throw new Error("Not implemented"); }`
Java	`.java`	`public T func(...) { throw new UnsupportedOperationException(); }`
C#	`.cs`	`public T Func(...) { throw new NotImplementedException(); }`

3. Mechanizm

1. **Walidacja wejścia** — finding musi być typu `missing`, musi mieć `requirement_id` i `requirement_text`.
2. **Konwersja nazwy wymagania na identyfikator** — `_normalize_identifier()`:
   - camelCase / snake_case w zależności od języka
   - usunięcie stop-słów ("System must", "shall")
   - dla Java/C#: PascalCase
3. **Render szablonu Jinja2** — `templates/{language}.j2`
4. **Walidacja składni** — tree-sitter parse na wyjściu
5. **Zwrócenie** `SkeletonResponse` z `code`, `language`, `file_name`, `skeleton_path`

4. Szablony Jinja2

Lokalizacja: `backend/app/services/gap_analysis/templates/`

Przykład (`python.j2`):

```
"""TODO: {{ requirement_text }}

Requirement ID: {{ requirement_id }}
Priority: {{ priority }}
Generated from gap analysis — do not edit this header.
"""

def {{ function_name }}({{ params }}):
    """{{ requirement_text }}"""
    raise NotImplementedError(
        "Skeleton for {{ requirement_id }} — implement per spec."
    )
```

5. Walidacja tree-sitter

Po wyrenderowaniu szablonu, generator wywołuje tree-sitter aby upewnić się, że wynik jest poprawnym kodem w danym języku. Błędy składni są rzadkie (szablony są starannie sformułowane), ale walidacja daje gwarancję, że użytkownik dostaje kompilowalny / interpretowalny kod.

6. Persystencja

Pole `gap_findings.skeleton_path` przechowuje ścieżkę do wygenerowanego pliku skeletonu (migracja `025_add_skeleton_path_to_gap_findings.py`). Generator zapisuje plik pod:

```
{project_root}/.testgen/skeletons/{requirement_id}.{ext}
```

7. Endpointy API

- `POST /projects/{project_id}/gap-findings/{finding_id}/skeleton` — generuje skeleton dla danej luki
- `GET /projects/{project_id}/gap-findings/{finding_id}/skeleton` — zwraca istniejący skeleton

8. Frontend

UI: przycisk "Generuj skeleton" w wierszu tabeli findings dla `gap_type=missing`. Modal z podglądem kodu (CodeMirror / Monaco), przycisk "Kopiuj do schowka" + "Pobierz plik".

9. Testowanie

Testy jednostkowe:
`backend/tests/unit/services/gap_analysis/test_skeleton_generator.py` — pokrywają 4 języki, normalizację nazw, walidację tree-sitter.

10. Powiązane dokumenty

- 05 — Recommendation engine
- 03 — Klasyfikator luk

Integracje z trackerami (Jira, Azure DevOps)

> Pliki:
>
> - `backend/app/models/integration.py`
> - `backend/app/services/integrations/jira_service.py`
> - `backend/app/services/integrations/azure_devops_service.py`
> - `backend/app/api/routes/integrations.py`

1. Odpowiedzialność

Moduł integracji synchronizuje znaleziska gap analysis z zewnętrznymi trackerami zadań, aby umożliwić zespołom pracę nad lukami w istniejącym przepływie (workflow) bez konieczności ręcznego kopiowania informacji.

Dwóch dostawców:

- **Jira** (Cloud, REST API v3) — Story 5.8
- **Azure DevOps** (Cloud, Work Items API) — Story 5.9

2. Struktura danych

2.1 IntegrationConfig

```
class IntegrationConfig(Base, UUIDMixin, TenantMixin, TimestampMixin):
    __tablename__ = "integration_configs"
    project_id: UUID           # FK → projects.id
    provider: IntegrationProvider  # jira | azure_devops
    base_url: str              # np. https://test.atlassian.net
    encrypted_credentials: str  # Fernet token {email:api_token}
    settings: dict             # np. {"project_key": "ENG", "issue_type":
         "Task"}
    is_active: bool
```

2.2 IntegrationSyncLog

```
class IntegrationSyncLog(Base, UUIDMixin, TenantMixin, TimestampMixin):
    __tablename__ = "integration_sync_logs"
    integration_config_id: UUID
    provider: IntegrationProvider
    status: SyncStatus          # pending | running | success | failed |
        partial
    total_items: int
    synced_items: int
    failed_items: int
    error_message: str | None
    started_at: datetime
    finished_at: datetime | None
    external_ids: dict          # {gap_finding_id: external_issue_key}
```

3. Szyfrowanie poświadczeń

`encrypted_credentials` przechowuje token Fernet (symmetric encryption) zawierający JSON z polem `email` i `api_token`. Klucz przechowywany w zmiennej środowiskowej `INTEGRATION_ENCRYPTION_KEY` (lub `FERNET_KEY` jako fallback).

```
# Szyfrowanie przy zapisie
cipher_text = Fernet(key).encrypt(json.dumps({"email": ..., "api_token":
        ...}).encode())

# Odszyfrowanie przy użyciu
creds = json.loads(Fernet(key).decrypt(cipher_text))
```

4. JiraService

> Plik: `backend/app/services/integrations/jira_service.py`

4.1 Endpointy API

- `POST /serverinfo` — test połączenia
- `POST /issue` — tworzenie issue
- `GET /issue/{key}` — weryfikacja

4.2 Format issue

```
{
  "fields": {
    "project": {"key": "ENG"},
    "summary": "[TestGEN] Missing: REQ-ANL-004 — export do PDF",
    "description": "...",
    "issuetype": {"name": "Task"},
    "priority": {"name": "High"},
    "labels": ["testgen", "gap-analysis"]
  }
}
```

Priorytet mapowany z gap severity:

- `critical` → "Highest"
- `warning` → "High"
- `info` → "Medium"

5. AzureDevOpsService

> Plik: `backend/app/services/integrations/azure_devops_service.py`

5.1 Endpointy API

- Uwierzytelnianie: PAT (Personal Access Token) przez Basic Auth
- `POST {org}/{project}/_apis/wit/workitems/$Task?api-version=7.0` — JSON Patch

5.2 Format work item

Azure DevOps używa JSON Patch (operacje `add`):

```
[
  {"op": "add", "path": "/fields/System.Title", "value": "..."},
  {"op": "add", "path": "/fields/System.Description", "value": "..."},
  {"op": "add", "path": "/fields/Microsoft.VSTS.Common.Priority",
         "value": 2},
  {"op": "add", "path": "/fields/System.Tags", "value": "testgen; gap-
         analysis"}
]
```

Priorytet mapowany 1–4 z gap severity.

6. Sync flow

1. Użytkownik konfiguruje integrację (URL + email + token) w Settings → Integrations.

2. Test połączenia (`POST /integrations/{provider}/test`).
3. W widoku gap-analysis wybiera findings i klika "Sync do Jira/ADO".
4. Backend tworzy `IntegrationSyncLog(status="pending")` i asynchronicznie:
   - Iteruje po findings
   - Dla każdego tworzy issue (lub pomija jeśli już istnieje external_id)
   - Aktualizuje `external_ids` map
   - Na koniec: `status="success"` (lub `"partial"` / `"failed"`)
5. UI pokazuje progress + wynik.

7. Deduplikacja

Pole `external_ids` w `IntegrationSyncLog` przechowuje mapę `{gap_finding_id: external_issue_key}`. Ponowny sync pomija findings, które już mają external_key, chyba że użytkownik wymusi overwrite.

8. Endpointy API

- `POST /projects/{project_id}/integrations/{provider}/configure` — zapis konfiguracji
- `POST /projects/{project_id}/integrations/{provider}/test` — test połączenia
- `POST /projects/{project_id}/integrations/{provider}/sync` — uruchomienie sync
- `GET /projects/{project_id}/integrations/sync-logs` — historia logów

9. Frontend

UI: `frontend/src/app/features/integrations/` — formularz konfiguracji (URL, email, token, project_key), test connection button, lista sync logs z statusem i licznikami (synced/failed).

10. Bezpieczeństwo

- Poświadczenia szyfrowane Fernet (at-rest)
- RLS per-tenant — integracja jest dostępna tylko dla członków tenant
- WALIDACJA URL: `base_url` musi być HTTPS w produkcji
- Rate limiting: max 10 syncs na godzinę per projekt (zapobieganie pętlom)

11. Testowanie

Testy jednostkowe: `backend/tests/unit/services/integrations/` (JiraService, AzureDevOpsService — z httpx MockTransport). Testy integracyjne: `backend/tests/integration/test_integrations_routes.py`.

12. Powiązane dokumenty

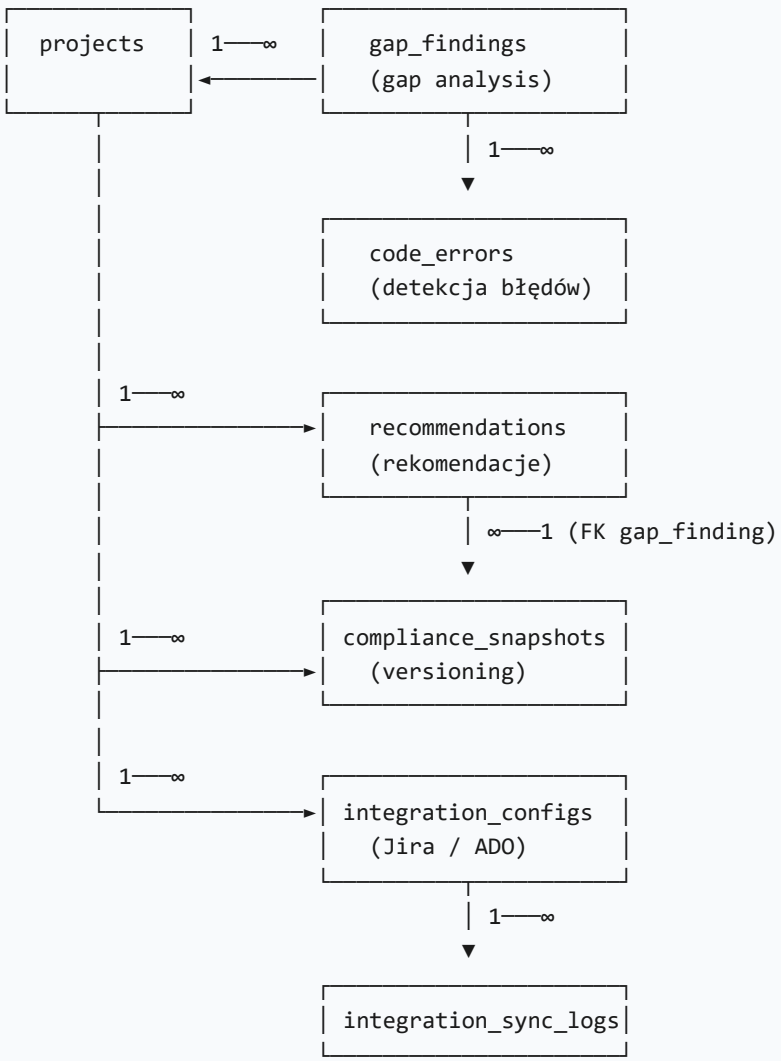
- 01 — Przegląd

- 02 — Przewodnik — integracje

Modele danych (ERD)

> Pliki: `backend/app/models/gap_finding.py`, `code_error.py`, `recommendation.py`, `compliance_snapshot.py`, `integration.py`

1. Diagram



2. gap_findings

Główna tabela modułu. Persystuje wszystkie znaleziska luk.

Kolumna	Typ	Opis
id	UUID (PK)	Klucz główny
tenant_id	UUID	RLS tenant
project_id	UUID FK	Projekt

Kolumna	Typ	Opis
analysis_id	UUID FK nullable	Powiązany AnalysisResult
requirement_id	str nullable	ID wymagania (np. REQ-ANL-004)
code_entity_id	str nullable	ID bytu kodu (z AnalysisResult)
gap_type	Enum	missing, different, extra, inconsistent
description	Text	Opis luki
requirement_text	Text nullable	Treść wymagania
code_reference	str(500) nullable	file:line
match_score	Float	0–100
match_method	str(30)	lev, jw, sem, combined
context	JSONB	excerpt + alternatywne dopasowania
severity	str(10)	info, warning, critical
source	str(10)	pochodzenie znaleziska (Story 5.10)
skeleton_path	str(500) nullable	Path do skeleton (Story 5.6)
created_at, updated_at	DateTime	Timestamps

Indeksy: project_id, gap_type, match_score, analysis_id, severity, source **RLS**: tenant_id

3. code_errors

Błędy kodu wykryte przez detektory (Story 5.4).

Kolumna	Typ	Opis
id	UUID PK	
project_id, tenant_id	UUID	RLS + scope
code_entity_id	str	Powiązany byt
file_path, line_start, line_end	str, int, int	Lokalizacja
error_category	Enum	syntax, logic, semantic, conformance
error_message	Text	Komunikat
severity	str(10)	info, warning, critical
suggested_fix	Text nullable	Sugerowana naprawa
context	JSONB	Ast / token info

4. recommendations

Rekomendacje naprawcze (Story 5.5).

Kolumna	Typ	Opis
id	UUID PK	

Kolumna	Typ	Opis
project_id, tenant_id	UUID	RLS + scope
gap_finding_id	UUID FK	Powiązana luka
priority	str(3)	P1, P2, P3, P4
confidence	Float	0.0–1.0
suggested_action	Text	Konkretna akcja
effort_estimate	str(2)	S, M, L
source	str(10)	pochodzenie rekomendacji
status	str(10)	open, accepted, applied, dismissed

5. compliance_snapshots

Punkt w czasie compliance (Story 5.7) — służy do porównań wersji.

Kolumna	Typ	Opis
id	UUID PK	
project_id, tenant_id	UUID	RLS + scope
snapshot_date	DateTime	Moment snapshotu
project_score	Float	Compliance score 0–100
module_scores	JSONB	{file_path: score}
finding_counts	JSONB	{missing: N, different: N, ...}
analysis_id	UUID FK nullable	Powiązany AnalysisResult

6. integration_configs + integration_sync_logs

Konfiguracja i logi sync do Jira/Azure DevOps (Stories 5.8/5.9). Szczegóły w 08 — Integracje.

7. Migracje

Migracja	Story	Zakres
`022_add_gap_findings_table.py`	5.1	Tabela gap_findings + RLS
`023_add_code_errors_table.py`	5.4	Tabela code_errors
`024_add_recommendations_table.py`	5.5	Tabela recommendations
`025_add_skeleton_path_to_gap_findings.py`	5.6	Kolumna skeleton_path
`026_add_compliance_snapshots_table.py`	5.7	Tabela compliance_snapshots
`027_add_integration_tables.py`	5.8/5.9	integration_configs + sync_logs
`035_add_source_to_gap_findings.py`	5.10	Kolumna source

8. RLS

Wszystkie tabele dziedziczą `TenantMixin` — wymusza `tenant_id` w zapytaniach przez PostgreSQL Row-Level Security. Polityki RLS są dodawane w migracjach per-tabela (wzorzec Epic 4).

9. Powiązane dokumenty

- 01 — Przegląd
- 08 — Integracje